Основы глубинного обучения

Лекция 7

Архитектуры свёрточных сетей

Евгений Соколов

esokolov@hse.ru

НИУ ВШЭ, 2026

VGG (2014)

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* **& Andrew Zisserman+**
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen,az}@robots.ox.ac.uk

https://arxiv.org/pdf/1409.1556.pdf

VGG (2014)

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG (2014)

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

https://arxiv.org/pdf/1409.1556.pdf

VGG (2014)

- Только маленькие свёртки 3×3
  - две подряд покрывают поле 5×5, три — 7×7, но параметров меньше
  - два слоя: $2 \cdot 9C^2 = 18C^2$ против $25C^2$ (при C каналах на входе и выходе каждого слоя)
  - три слоя: $3 \cdot 9C^2 = 27C^2$ против $49C^2$
  - +больше нелинейностей между ними
- Сеть собрана из одинаковых блоков по простому правилу (несколько 3×3 + pool, каналы ×2 после каждого pool: 64 → 128 → 256 → 512) — первая архитектура-конструктор, все следующие строятся аналогично
- Градиентный спуск с инерцией, dropout для двух первых полносвязных слоёв
- Хитрая инициализация: сначала обучается вариант A со случайными весами, потом им инициализируются более глубокие сети

https://arxiv.org/pdf/1409.1556.pdf

VGG (2014)

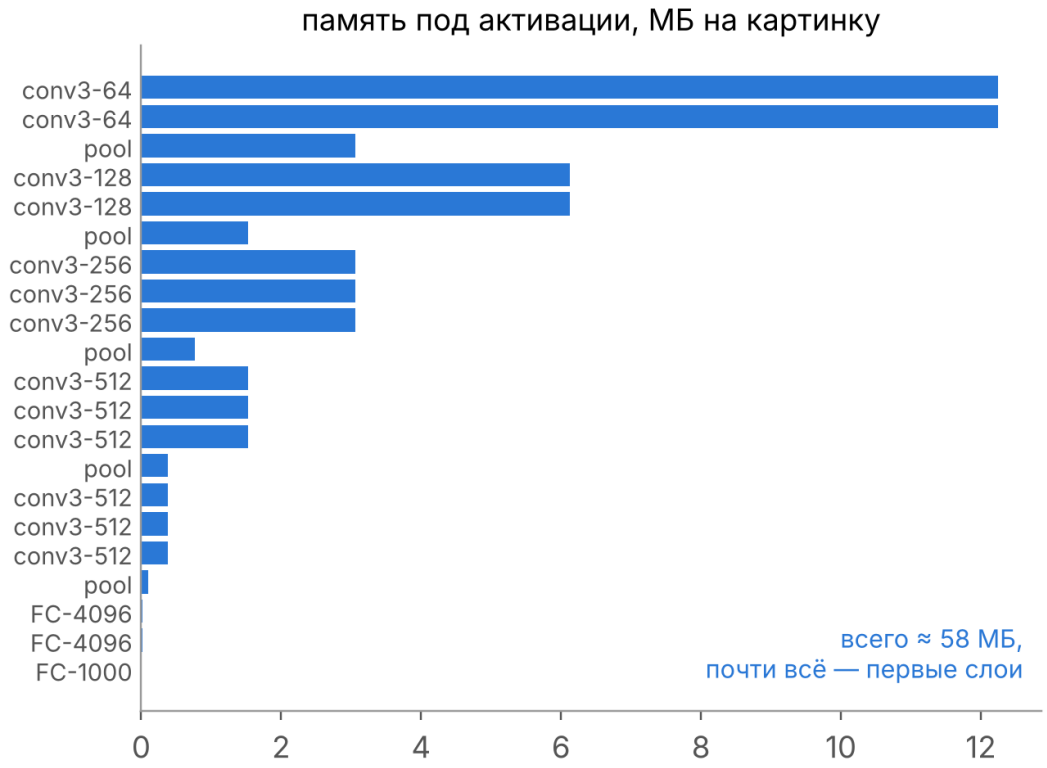
Table 3: **ConvNet performance at a single test scale.**

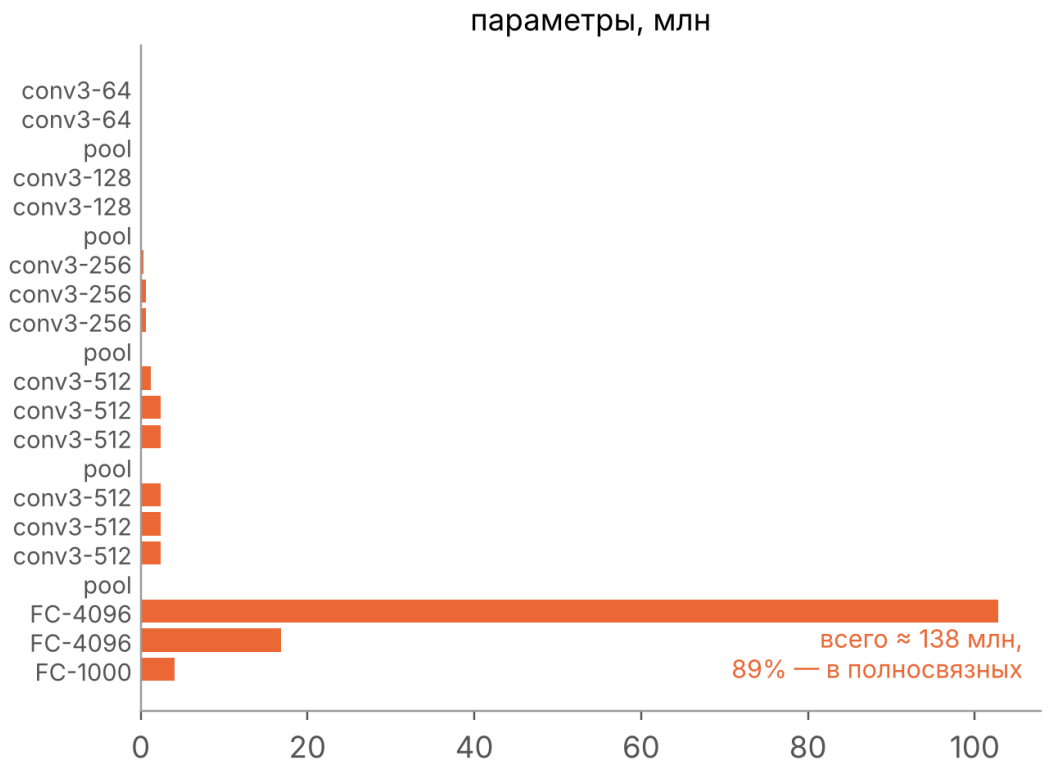
ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	**25.5**	**8.0**

https://arxiv.org/pdf/1409.1556.pdf

VGG-16: параметры и память

- 138 млн параметров, из них 89% — в трёх полносвязных слоях
- Память под активации почти вся уходит на первые свёрточные слои
- Вывод: убрать полносвязные слои и сжимать каналы дешёвыми свёртками





GoogLeNet (2014)

Going Deeper with Convolutions

Christian Szegedy[1], Wei Liu[2], Yangqing Jia[1], Pierre Sermanet[1], Scott Reed[3],
Dragomir Anguelov[1], Dumitru Erhan[1], Vincent Vanhoucke[1], Andrew Rabinovich[4]
[1]Google Inc. [2]University of North Carolina, Chapel Hill
[3]University of Michigan, Ann Arbor [4]Magic Leap Inc.
[1]{szegedy,jiayq,sermanet,dragomir,dumitru,vanhoucke}@google.com
[2]wliu@cs.unc.edu, [3]reedscott@umich.edu, [4]arabinovich@magicleap.com

http://arxiv.org/abs/1409.4842

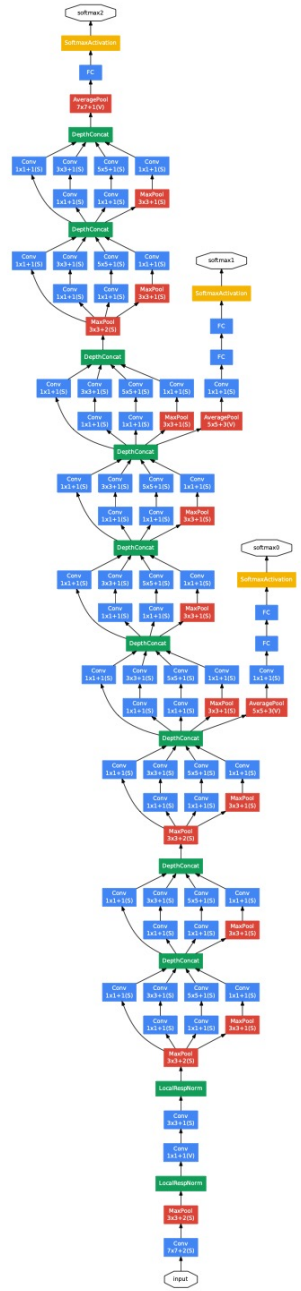
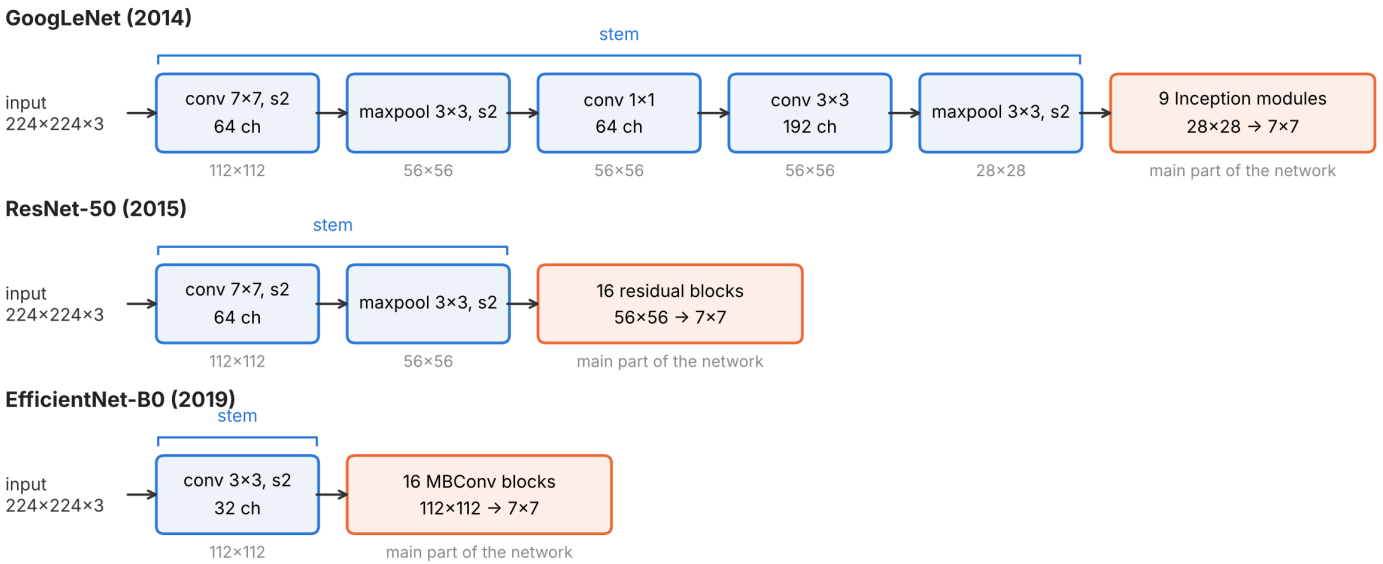


Figure 3: GoogLeNet network with all the bells and whistles.

Stem (первые слои)

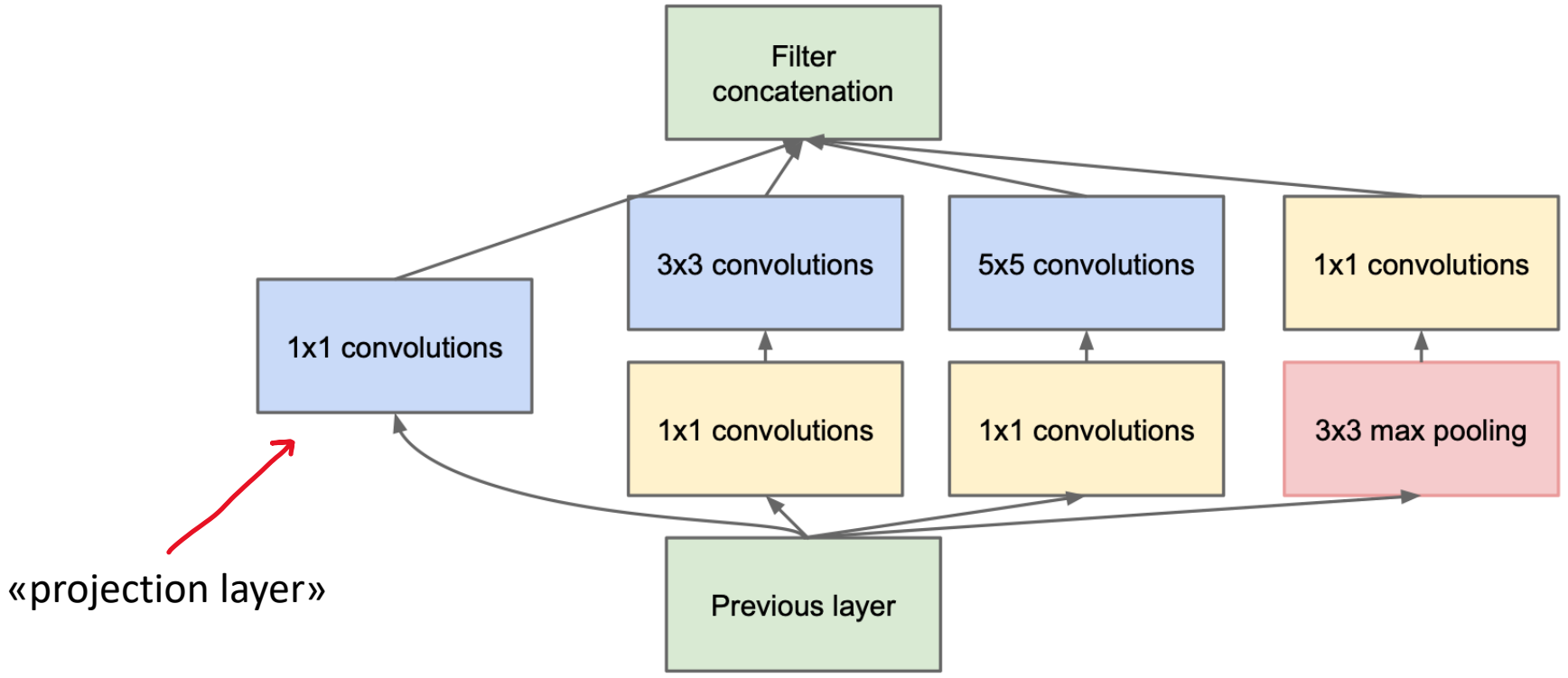
- Несколько обычных свёрток со страйдами (и пулингом) в самом начале сети. За 1-5 слоёв разрешение падает в 4 раза (224 → 56), число каналов растёт с 3 до 64–192.
- Зачем: стоимость слоя пропорциональна H·W, и на полном разрешении модуль с несколькими параллельными ветками слишком дорог по вычислениям и памяти
- Теперь стандартная часть любой архитектуры: ResNet — 7×7/2 + pool, Inception-v3 — три 3×3 вместо 7×7, EfficientNet — одна 3×3/2, ConvNeXt — 4×4/4



stem: a few plain strided convolutions at full resolution — resolution ÷4 (÷2 in B0), channels 3 → 64–192 — before the first expensive module; cost of a layer grows with H·W

https://arxiv.org/abs/1409.4842

GoogLeNet (2014)



(b) Inception module with dimensionality reduction

свёртки делаются с паддингом!

http://arxiv.org/abs/1409.4842

GoogLeNet (2014)

- 22 слоя, всего ~7 млн параметров — почти на порядок меньше AlexNet и в 20 раз меньше VGG-16
- Снижается число каналов перед «тяжёлыми» свёртками: свёртка 1×1 как дешёвое сжатие по каналам (в модуле операций в 2.4 раза меньше)
- Вместо полносвязных слоёв в конце — global average pooling
- Несколько выходных слоёв для улучшения обучаемости (вспомогательные классификаторы на промежуточных слоях)
- Обучается градиентным спуском с инерцией; top-5 ошибка 6.67% на ImageNet (ансамбль из 7 сетей)

http://arxiv.org/abs/1409.4842

Inception-v2/v3 (2015)

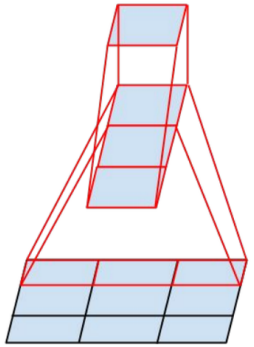
- Версии: GoogLeNet = v1 (сентябрь 2014); v2 и v3 — одна статья «Rethinking the Inception Architecture» (декабрь 2015): v2 — факторизованные свёртки, v3 — v2 плюс RMSProp, label smoothing, факторизованная 7×7 в stem, BatchNorm во вспомогательной голове
- Вспомогательные классификаторы: $L = L_0 + 0.3L_1 + 0.3L_2$ — в GoogLeNet введены, чтобы донести градиент до нижних слоёв. Проверка через год:
  - кривые обучения с головами и без них совпадают почти до конца; выигрыш — только чуть более высокое плато в самом конце
  - нижнюю голову можно убрать без потерь, а BatchNorm или dropout в голове улучшают основной классификатор ⇒ головы работают как регуляризатор
- top-1 ошибка: GoogLeNet 29.0 → BN-Inception 25.2 → факторизация 23.4 → RMSProp 23.1 → label smoothing 22.8 → факторизованная 7×7 21.6

https://arxiv.org/abs/1512.00567

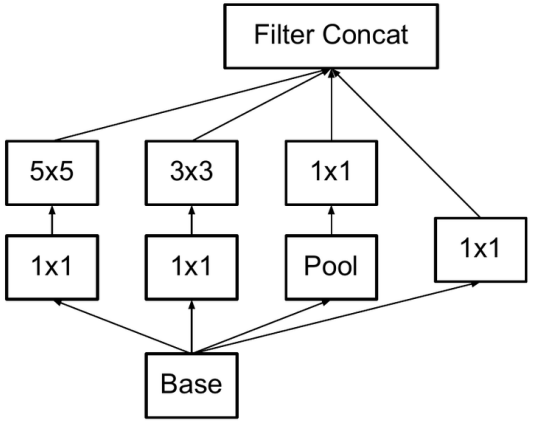
Факторизация свёрток в Inception-v3

- Внутри веток модуля: сначала 5×5 → две 3×3 (аргумент VGG), потом n×n → n×1 и 1×n: n^2 → $2n$ параметров (7×7: 49 → 14); с нелинейностью между ними лучше, чем без

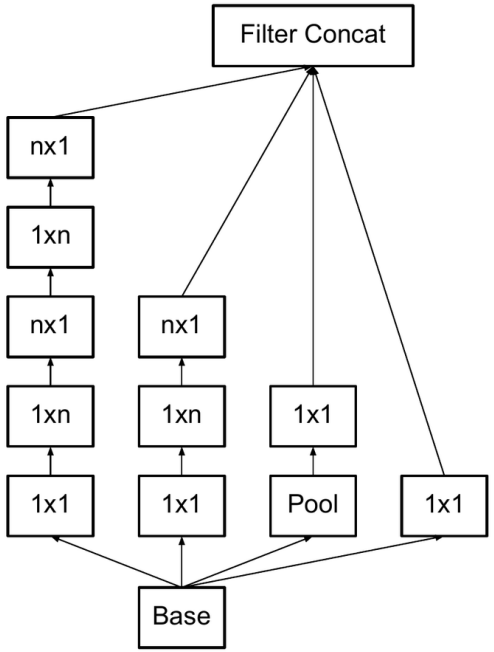
3×3 → 3×1 then 1×3 (Fig. 3)



GoogLeNet module (Fig. 4)



Inception-v3 module, 17×17 grid, n = 7 (Fig. 6)



https://arxiv.org/abs/1512.00567

Label smoothing

- Целевой вектор $(1-\varepsilon)\cdot[k=y]+\frac{\varepsilon}{K}$ вместо one-hot, ε = 0.1: правильному классу 0.92, остальным по 0.02 (K = 5)
- В статье: +0.2 пункта top-1 и лучше калибровка; стандартная часть рецепта обучения

	class 1	class 2	class 3 (true)	class 4	class 5
logits z	1.2	0.3	5.0	0.0	-0.5
softmax p	0.021	0.009	0.959	0.006	0.004
one-hot target q	0	0	1	0	0
smoothed target q′ (ε = 0.1)	0.02	0.02	0.92	0.02	0.02

https://arxiv.org/abs/1512.00567

ResNet (2015)

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

https://arxiv.org/abs/1512.03385

ResNet (2015)

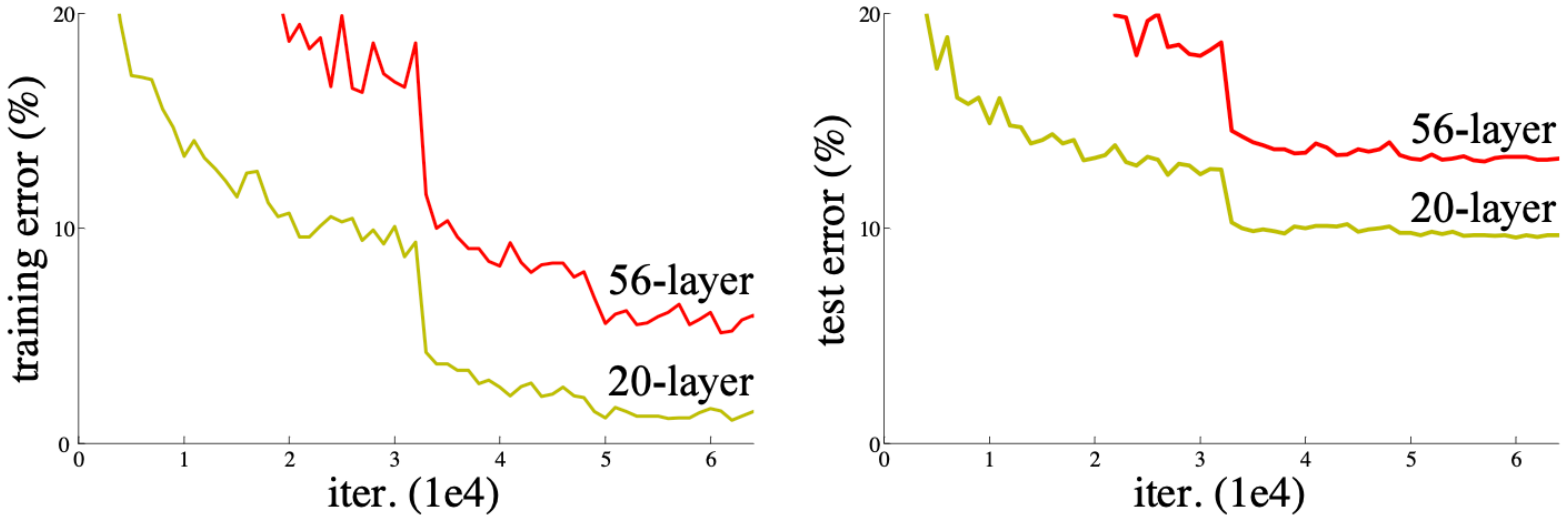
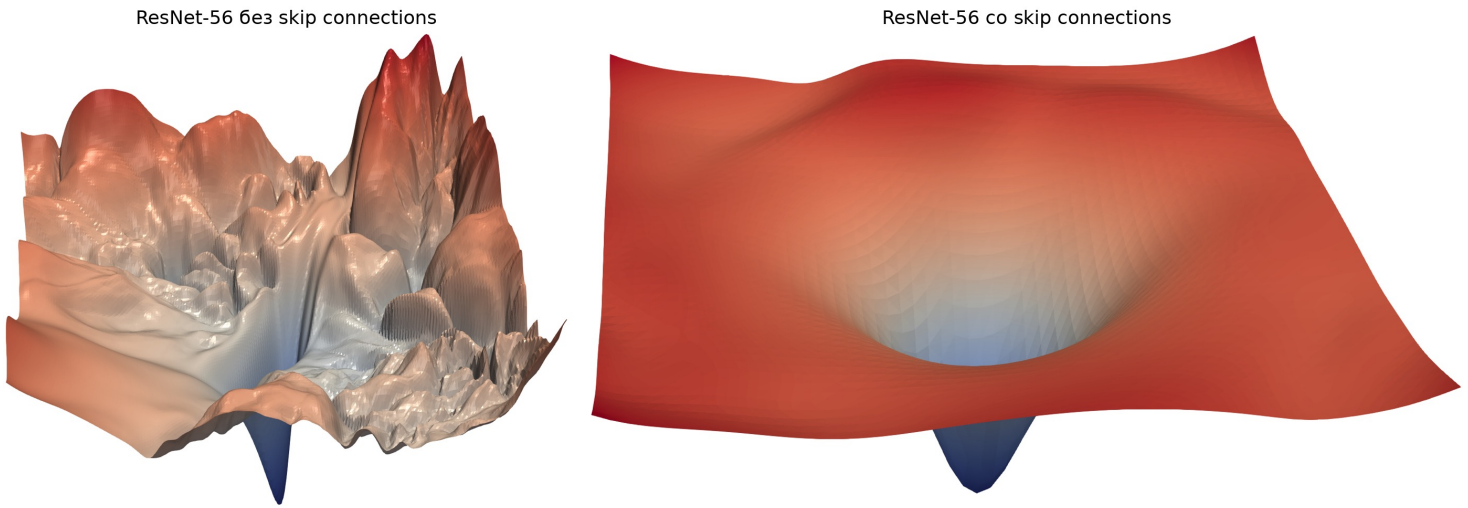


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

https://arxiv.org/abs/1512.03385

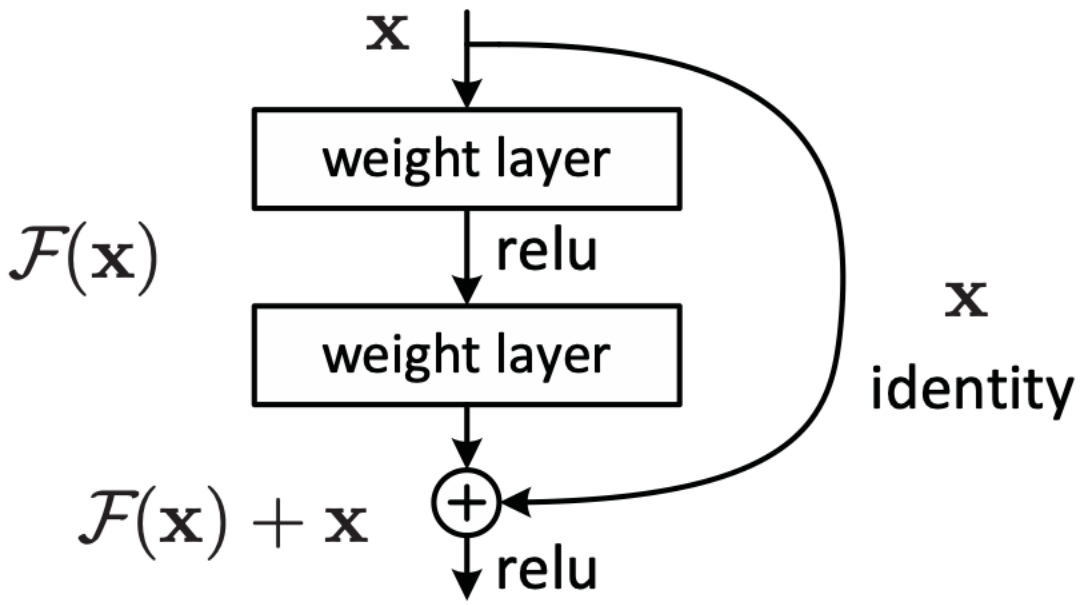
ResNet (2015)

- Добавление слоёв в свёрточную сеть ухудшает качество даже на обучении
- Хотя возможностей для переобучения больше, сеть почему-то не может ими воспользоваться
- Видимо, это проблема оптимизации: рельеф хаотичный, и BatchNorm сам по себе не спасает
- Идея: если глубокая сеть не может даже выучить тождественное отображение, дадим ей тождество бесплатно и будем учить только поправку к нему



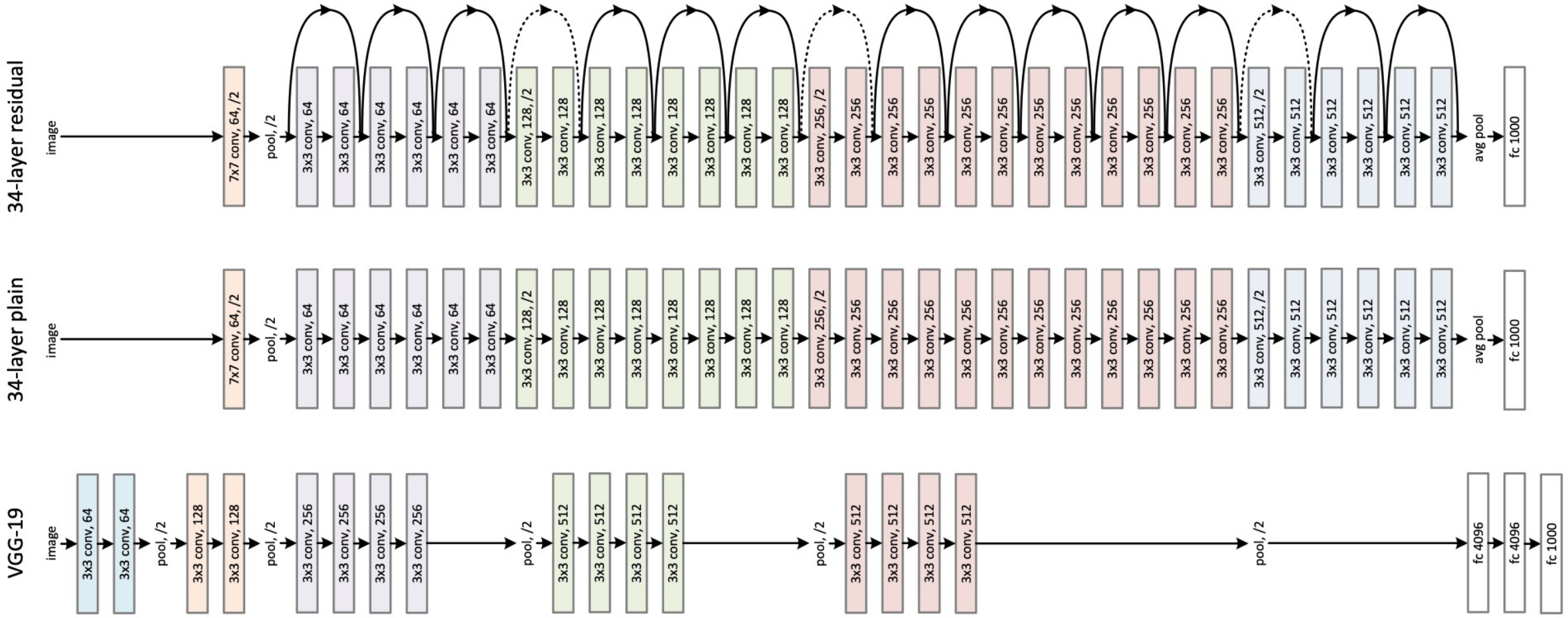
https://arxiv.org/abs/1512.03385

ResNet (2015)



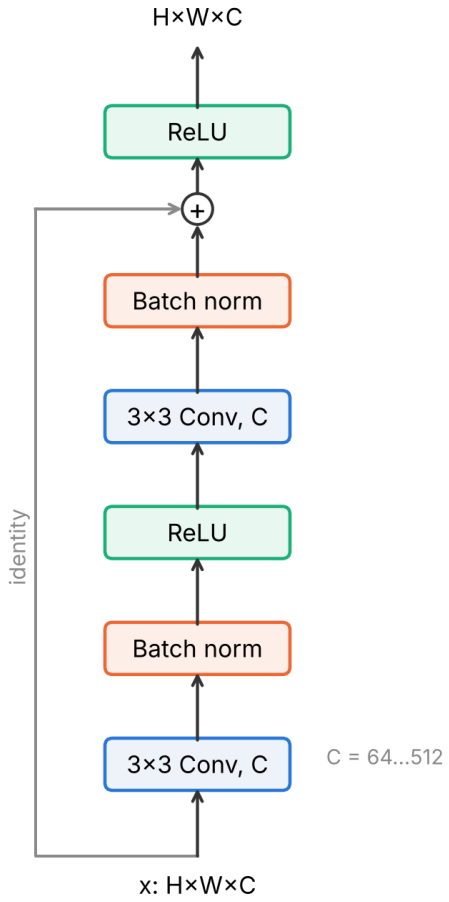
https://arxiv.org/abs/1512.03385

ResNet (2015)

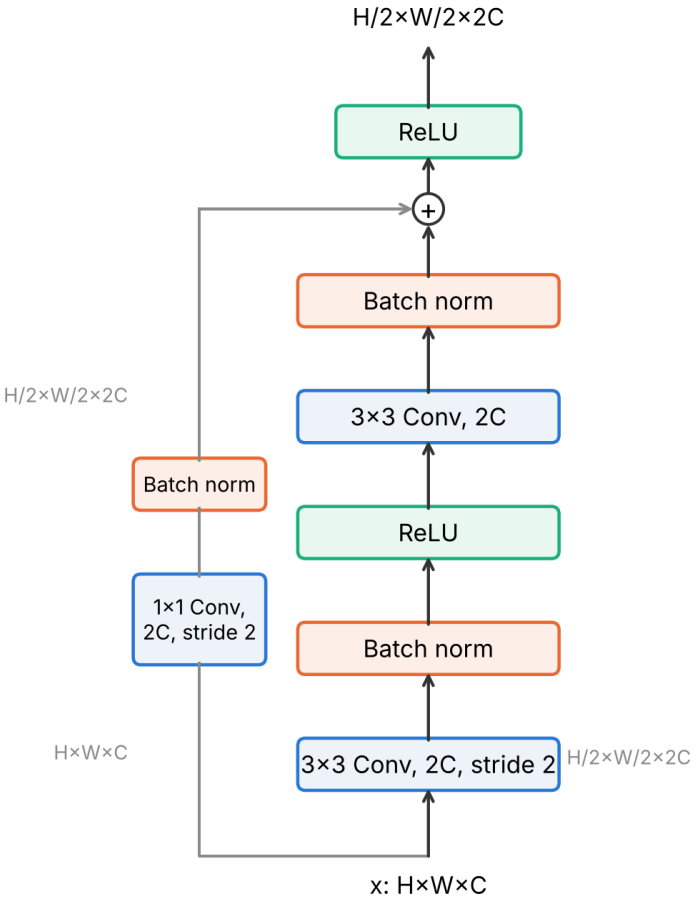

VGG-19
image
3x3 conv, 64
3x3 conv, 64
pool, /2
3x3 conv, 128
3x3 conv, 128
pool, /2
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
pool, /2
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
fc 4096
fc 4096
fc 1000
34-layer plain
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128
3x3 conv, 256
3x3 conv, 512
3x3 conv, 128, /2
3x3 conv, 256, /2
3x3 conv, 512, /2
avg pool
fc 1000
34-layer residual
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128
3x3 conv, 256
3x3 conv, 512
3x3 conv, 128, /2
3x3 conv, 256, /2
3x3 conv, 512, /2
avg pool
fc 1000

ResNet block

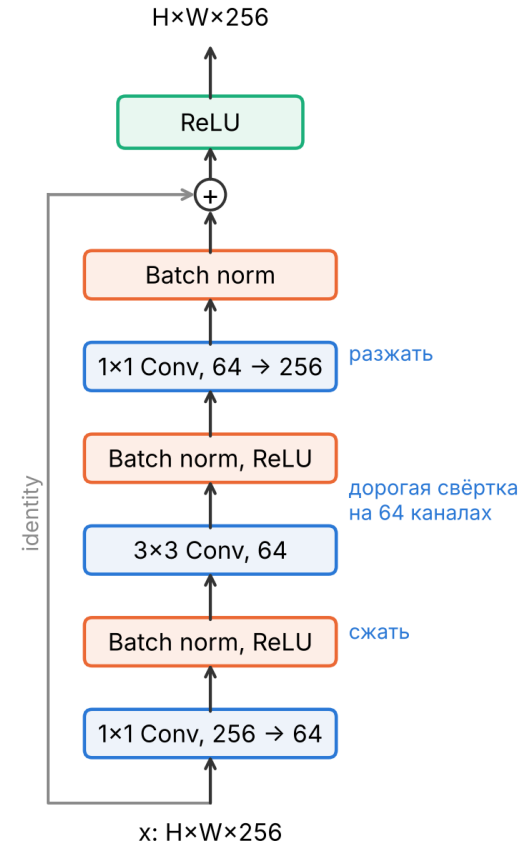
Basic block (ResNet-18/34)



Projection shortcut (смена размерности)



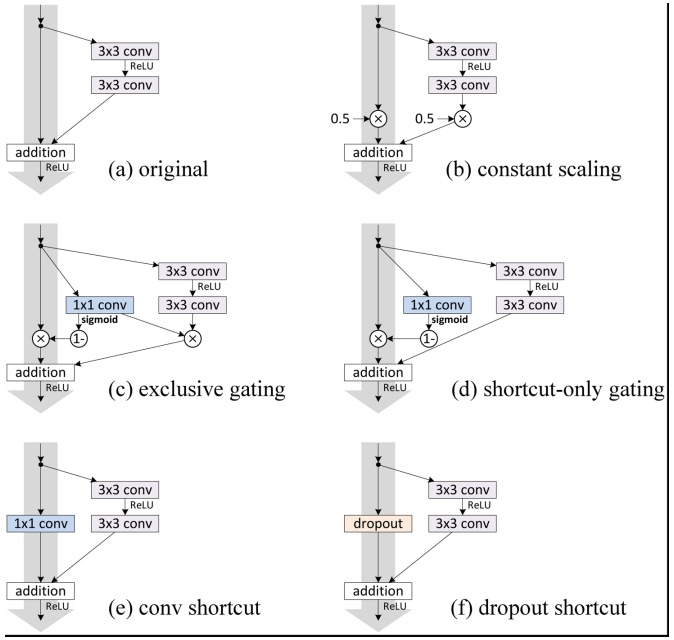
Bottleneck block (ResNet-50/101/152)

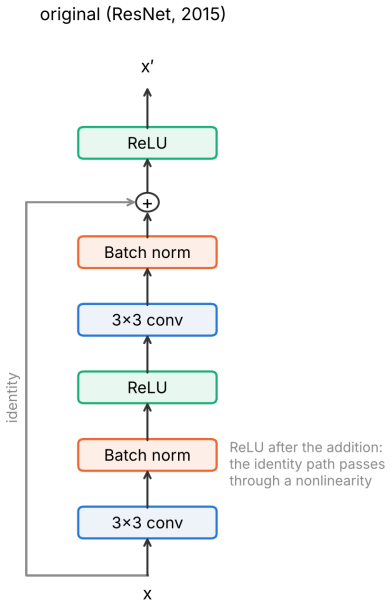


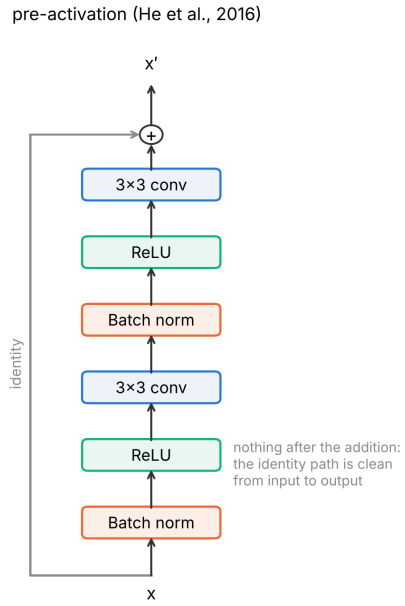
≈70 тыс. параметров вместо 1.2 млн у двух 3×3 на 256

Чем прямее обход, тем лучше

- He et al. 2016: что будет, если на обходе не тождество? ResNet-110, CIFAR-10, ошибка на тесте: тождество — 6.61%; умножение на 0.5 — 12.35%; гейт — 8.70%; свёртка 1×1 на обходе — 12.22%; dropout на обходе — не обучается
- Pre-activation: BatchNorm и ReLU перед свёртками, после сложения ничего — обход совсем чистый. ResNet-1001: 7.61 → 4.92% (CIFAR-10), 27.82 → 22.71% (CIFAR-100)
- Та же идея дальше: pre-norm в трансформерах — нормализация внутри ветки, остаточный путь нетронут

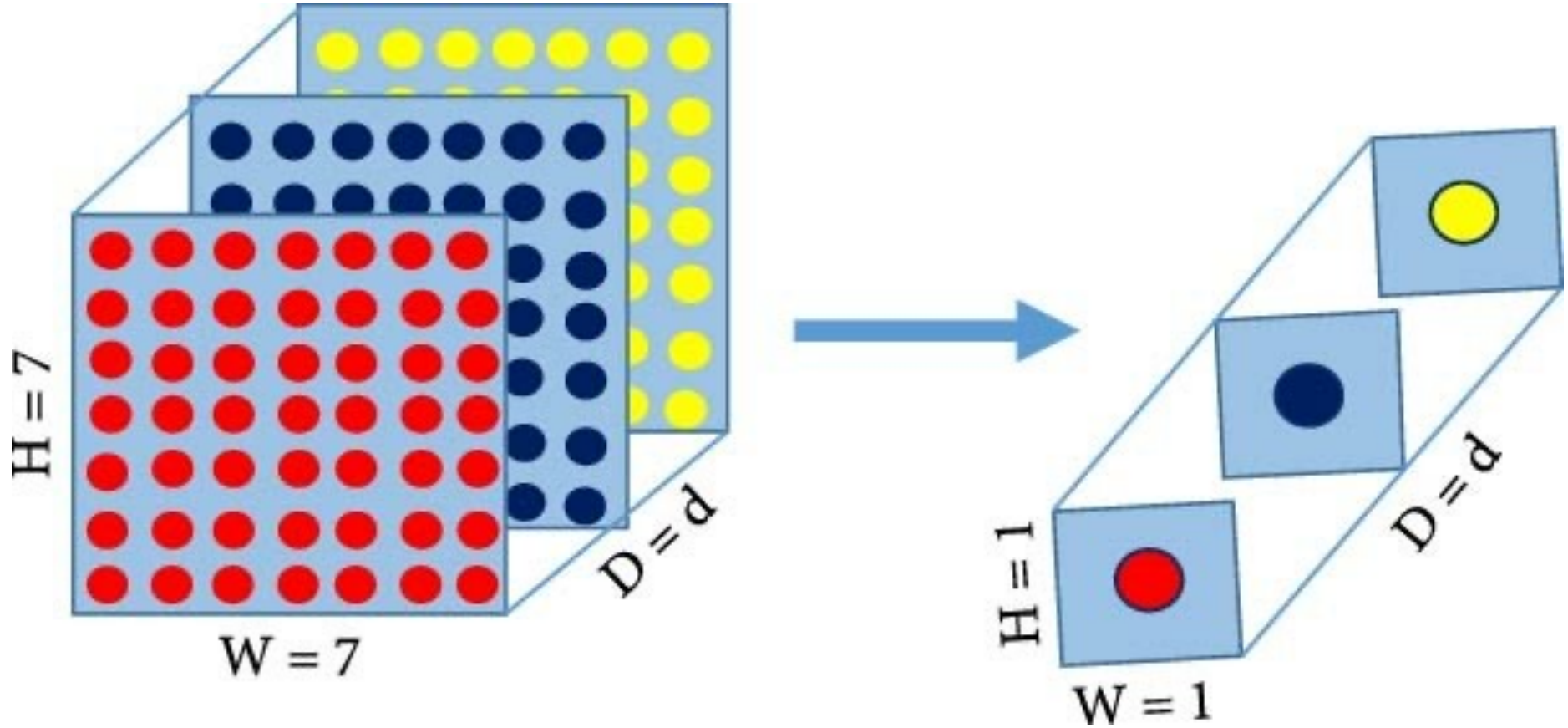






https://arxiv.org/abs/1603.05027

Global Average Pooling



Идея из Network in Network (Lin et al., 2013): усреднить каждую карту признаков в одно число. В NiN каналов ровно столько, сколько классов, и вектор идёт сразу в softmax; в GoogLeNet и ResNet после усреднения остаётся один небольшой полносвязный слой (2048 → 1000) вместо трёх огромных

https://arxiv.org/abs/1312.4400

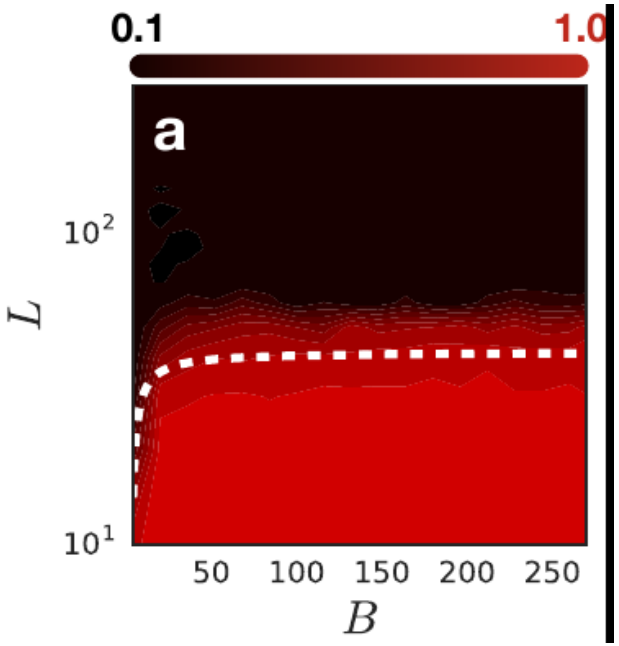
ResNet (2015)

- ResNet-50 и глубже: блок bottleneck 1×1 → 3×3 → 1×1 — сжать 256 каналов до 64, сделать дорогую 3×3 на 64, разжать обратно
  - 1×1, 256 → 64: 256·64 = 16 384 параметров (ядро 1×1 — просто матрица C×T)
  - 3×3, 64 → 64: 9·64·64 = 36 864; 1×1, 64 → 256: 64·256 = 16 384; итого ≈ 70 тыс.
  - одна 3×3 на 256 каналах — 9·256·256 = 590 тыс., basic block из двух таких — 1.2 млн: bottleneck в 17 раз дешевле блока, который заменяет
- Даёт низкую ошибку на обучении даже с 1000 слоёв (но там плохо на тестовой выборке)
- Обучается градиентным спуском с инерцией со случайной инициализацией, BatchNorm после каждой свёртки
- Global Average Pooling — позволяет работать с изображениями разного размера
- Ошибка 4.49% на ImageNet (ResNet-152, одна модель), 3.57% ансамблем — лучше человека
- ResNet-50 (25.6 млн параметров) — до сих пор стандартная модель и бейзлайн в исследованиях

https://arxiv.org/abs/1512.03385

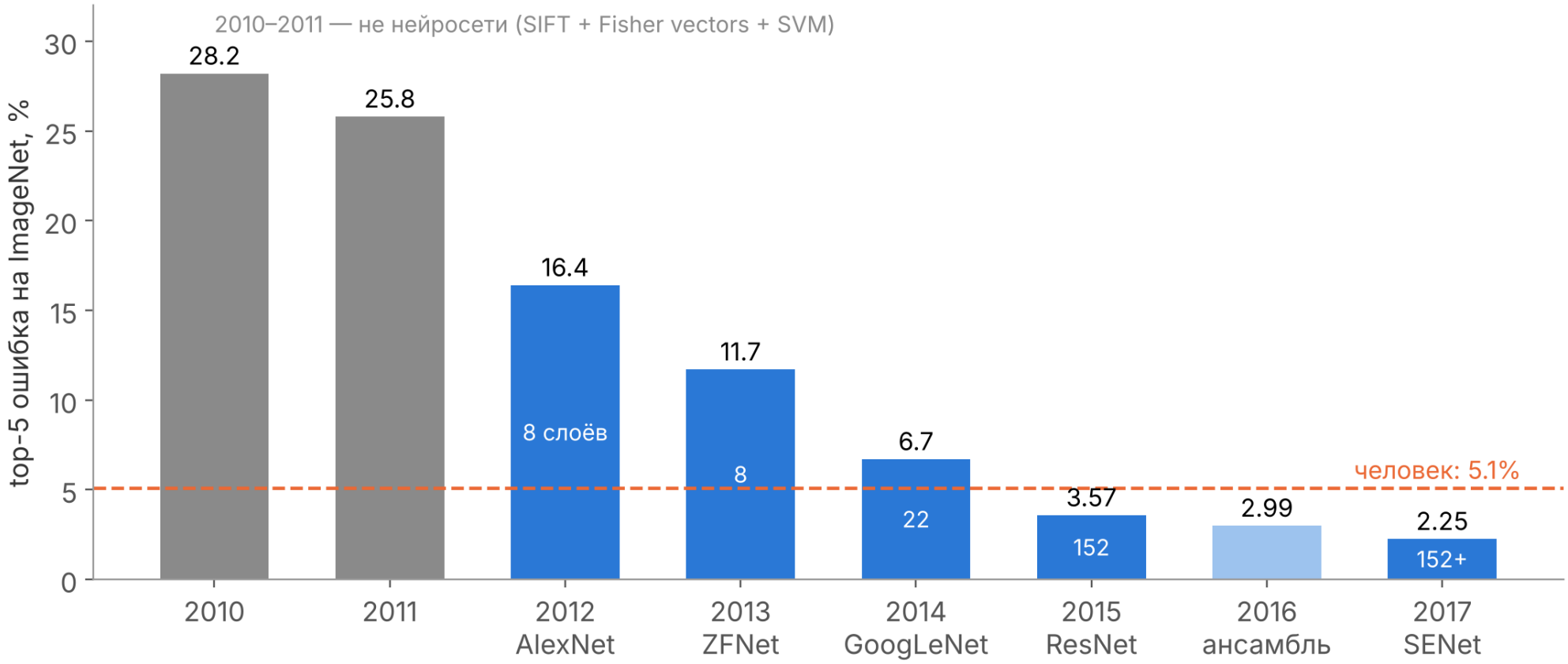
BatchNorm без skip connections

- Разберём первый шаг обучения — сразу после инициализации
- Выходы сети могут взорваться или обнулиться с глубиной; это чинится инициализацией, сохраняющей дисперсию (Xavier, He)
- BatchNorm инициализируется γ = 1, β = 0: на выходе каждого BatchNorm дисперсия ровно 1 — прямой проход не взрывается никогда
- ReLU поверх BatchNorm: выход ≥ 0, у него появляется ненулевое среднее. Следующий BatchNorm это среднее вычитает и растягивает остаток обратно до дисперсии 1 — в 1.47 раза. На обратном проходе градиент растягивается во столько же на каждом слое; число задаёт только ReLU.
- Итог: без residual-связей сеть с BatchNorm обучается примерно до 50 слоёв ($1.47^{50} \approx 2 \cdot 10^8$), глубже — уже нет
- На картинке: по вертикали глубина L, по горизонтали размер батча B, цвет — доля верных ответов на MNIST после 10 эпох (красное — обучилась, чёрное — нет), белый пунктир — предсказание теории
- Residual-блок: $x_{l+1} = x_l + f(x_l)$. По стволу сигнал и градиент проходят нетронутыми — множитель ровно 1; вклад ветки прибавляется, а не умножается



https://arxiv.org/abs/1902.08129

Победители ILSVRC



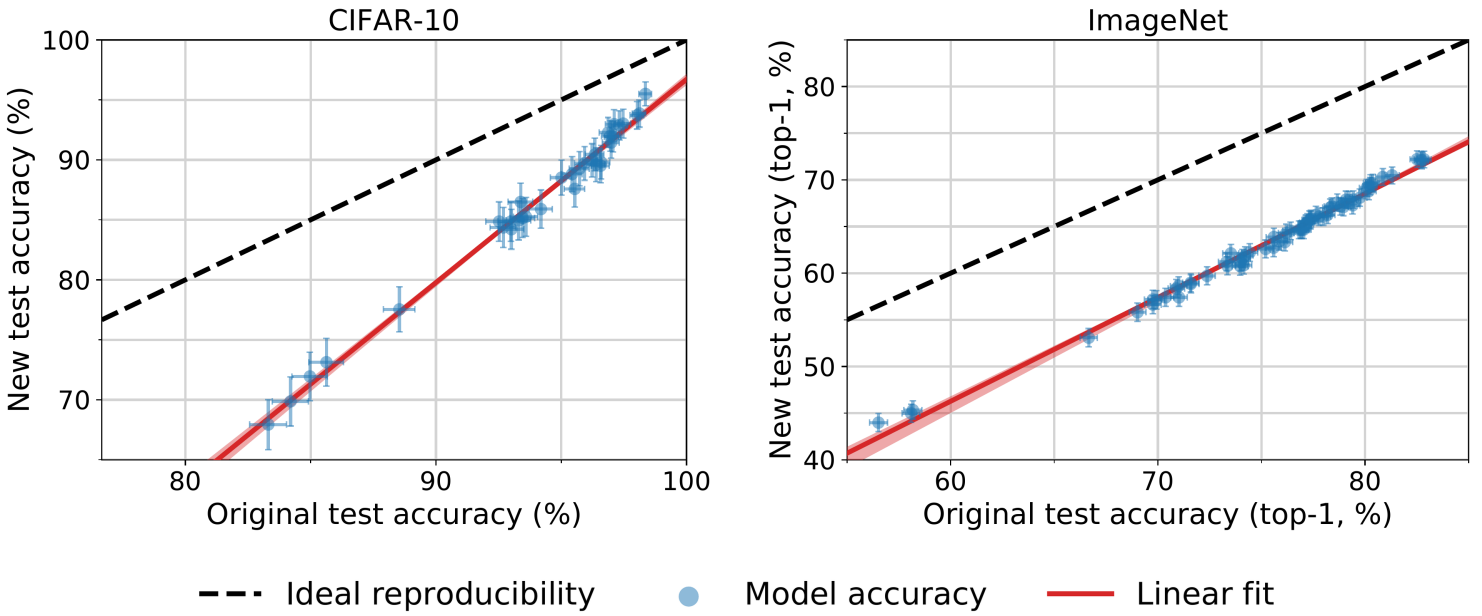
Почему человек ошибается в 5%

- 5.1% — один тренированный человек (Karpathy, 2014) на ~500 картинках, около минуты на каждую, с поиском по 1000 классам; второй разметчик без подготовки — 12%
- Классы очень мелкие: 120 пород собак, десятки грибов, змей, птиц — хаски от маламута без подготовки не отличить
- На картинке часто несколько объектов, а метка одна — человек называет не тот, что имели в виду разметчики
- Сами метки шумные: по поздним оценкам около 6% валидационных меток ошибочны или спорны (Northcutt et al., 2021; Beyer et al., 2020)

https://arxiv.org/abs/2006.07159

Обобщается ли на новый ImageNet?

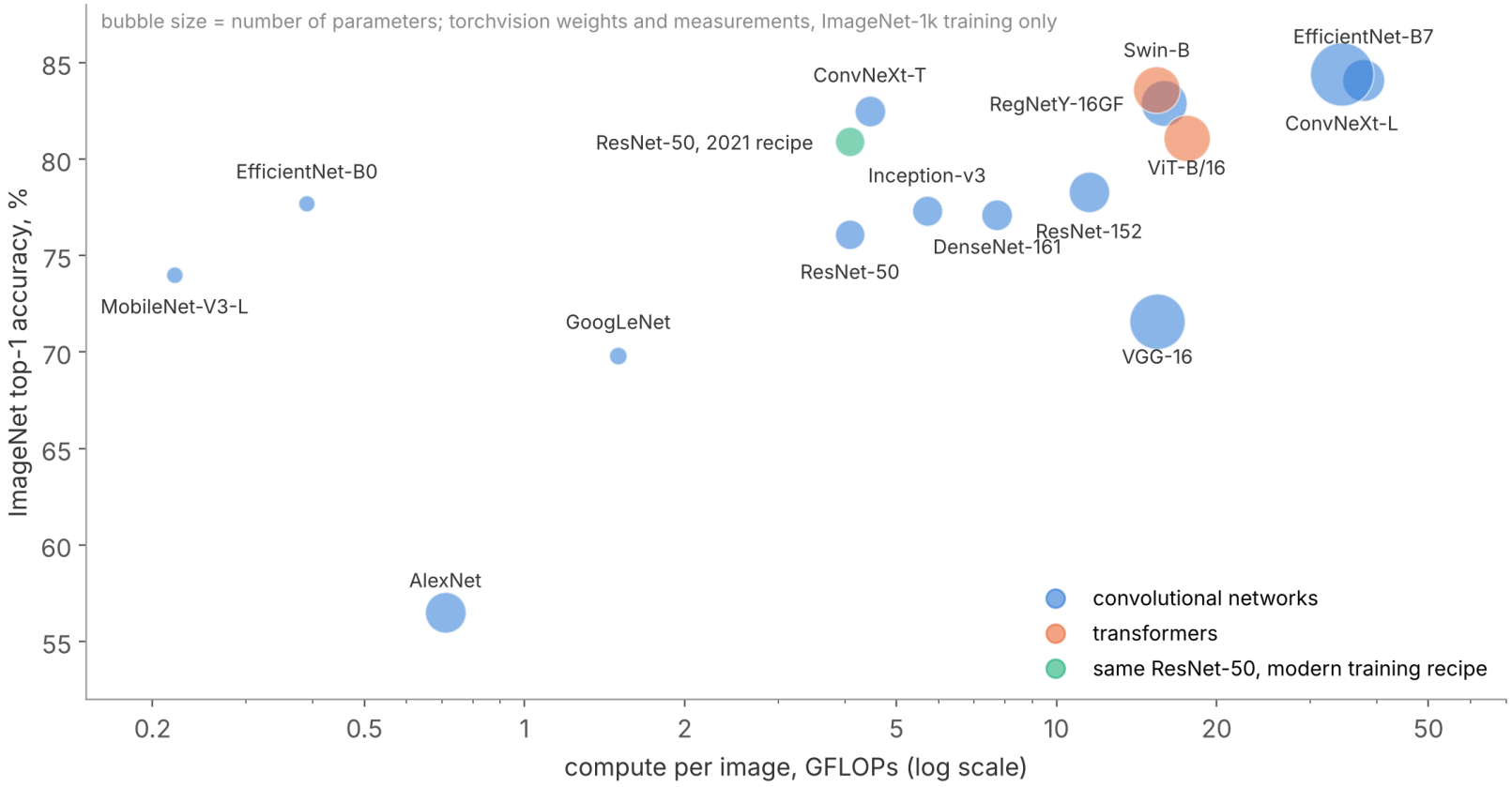
- Recht, Roelofs, Schmidt, Shankar (2019): собрали новые тестовые выборки по той же процедуре, что и оригинальные — CIFAR-10.1 и ImageNetV2
- Все модели проседают: 3–15 пунктов доли верных ответов на CIFAR-10, 11–14 на ImageNet
- Но порядок моделей сохраняется, а наклон прямой больше 1: пункт прогресса на старом тесте даёт больше пункта на новом
- Значит, это не переобучение на тестовую выборку, а скорее сдвиг распределения: даже аккуратное повторение сбора данных даёт другие картинки



https://arxiv.org/abs/1902.10811

После соревнования

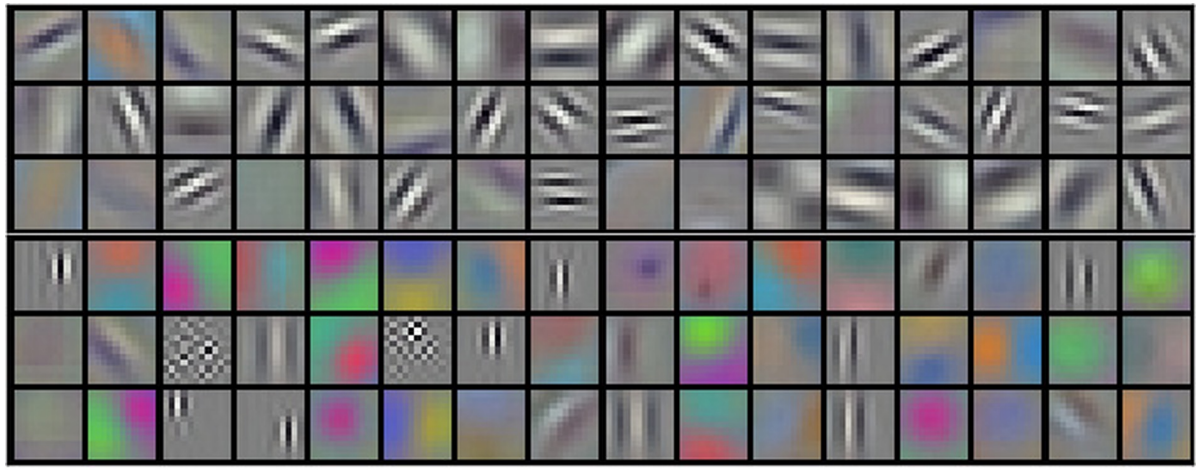
- Соревнование закончилось в 2017, но ImageNet остался главным бенчмарком. За десять лет: доля верных ответов выросла с 56 до 84%, эффективность — на порядки (EfficientNet-B0 обгоняет VGG-16 при в 40 раз меньших вычислениях)



https://docs.pytorch.org/vision/stable/models.html

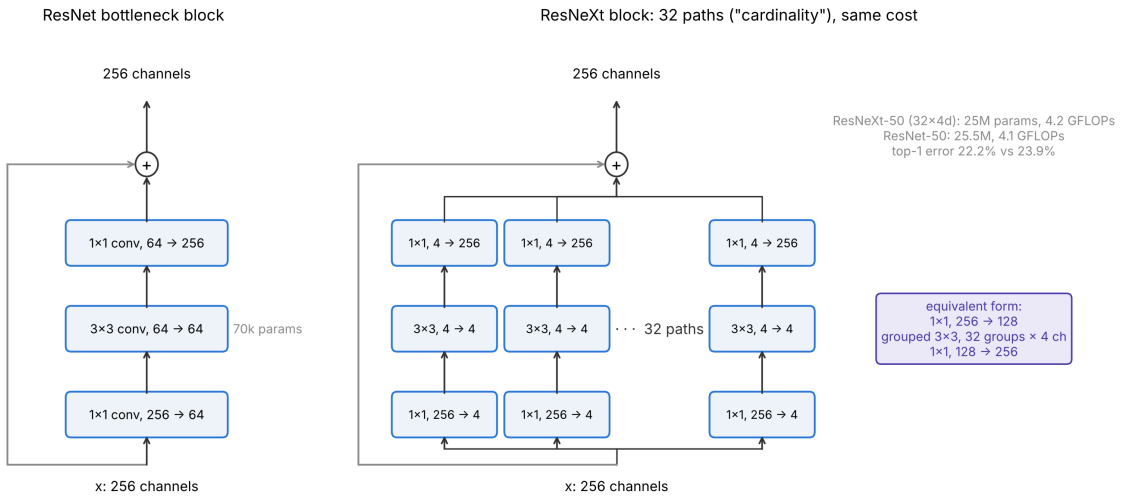
Групповая свёртка

- AlexNet (2012): свёртки разбиты на две группы, потому что сеть не влезала в одну GPU — и группы специализировались: на одной GPU фильтры чёрно-белые, на другой цветные
- Групповая свёртка: входные каналы делятся на G групп, каждое ядро видит только свою группу — параметров и операций в G раз меньше
- ResNeXt (2017): блок ResNet с 32 группами при тех же вычислениях — ResNeXt-50 на 1.7 пункта лучше ResNet-50 на ImageNet
- Предельный случай G = C: у каждого канала своё ядро — depthwise-свёртка; с 1×1 следом — depthwise separable: Xception, MobileNet



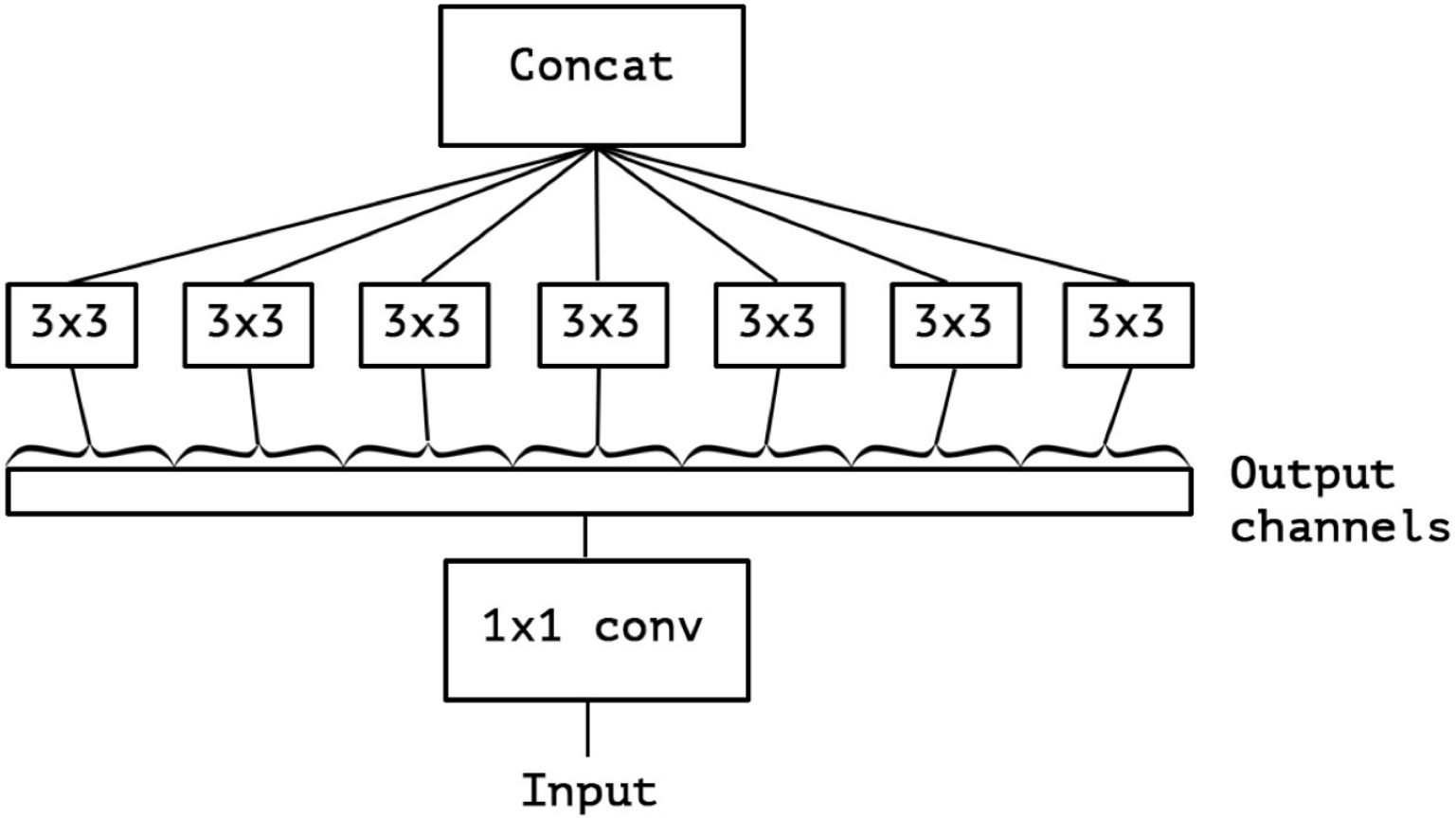
AlexNet, 96 first-layer kernels 11×11×3

top 48 trained on GPU 1 (grayscale, orientation/frequency), bottom 48 on GPU 2 (color)



https://arxiv.org/abs/1611.05431

Xception



https://arxiv.org/abs/1610.02357

Xception

- Depthwise separable convolution: свёртка 3×3 отдельно по каждому каналу (у каждого канала своё ядро), затем 1×1 по каналам
  - параметров: $9C + CT$ вместо $9CT$ у обычной 3×3 — отношение $\frac{1}{9} + \frac{1}{T}$, то есть в 8–9 раз меньше; операций — во столько же
- Разделяется роль свёрток: либо по пространству, либо по каналам
- Между пространственной и канальной свёртками нет нелинейности, с ней хуже

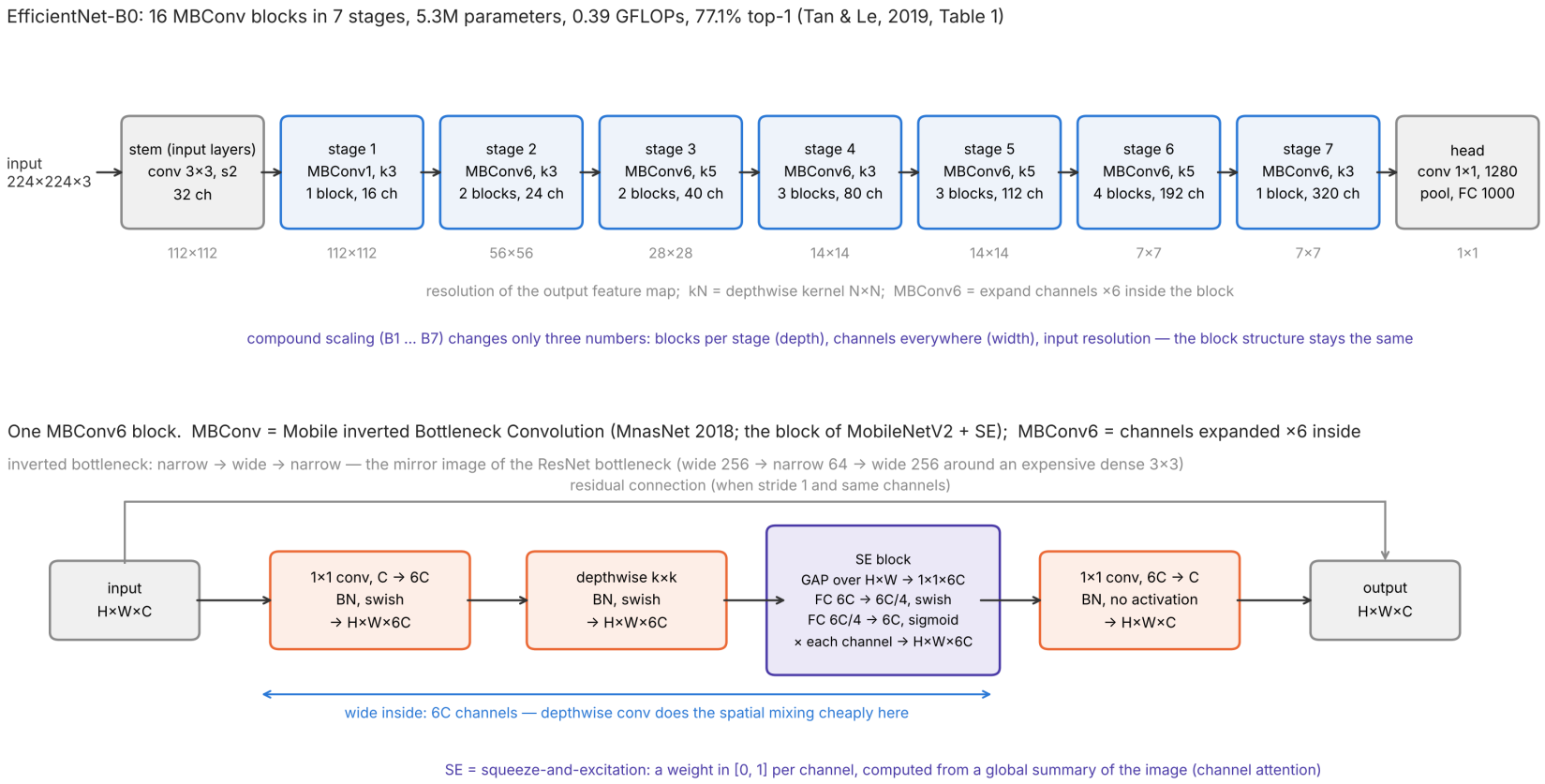
https://arxiv.org/abs/1610.02357

После ResNet

- ResNeXt (2017) — свёртки группами по каналам (grouped convolution): шире при тех же вычислениях
- DenseNet (2017) — каждый слой получает на вход выходы всех предыдущих
- SENet (2017) — что-то вроде механизма внимания, модель сама взвешивает каналы; победитель последнего ILSVRC
- MobileNet (2017) — depthwise separable свёртки, для телефонов
- EfficientNet (2019) — автоматический поиск базового блока + согласованное масштабирование глубины, ширины и разрешения
- RegNet (2020) — модель, полученная через поиск в пространстве архитектур
- ConvNeXt (2022) — ResNet, модернизированный по рецептам трансформеров

EfficientNet (2019): базовая сеть B0

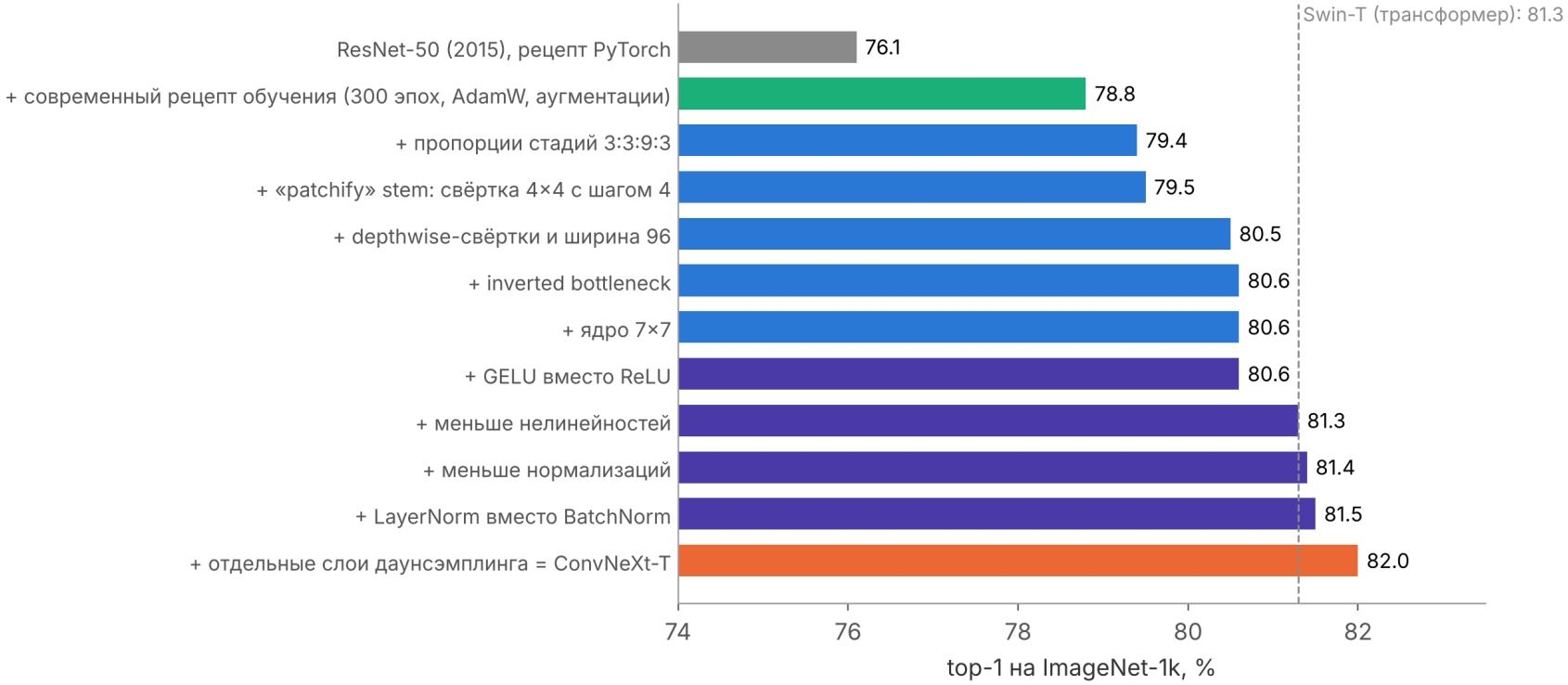
- B0 найдена автоматическим поиском архитектуры; 16 блоков одного типа между stem и головой



https://arxiv.org/abs/1905.11946

ConvNeXt (2022)

- Взяли ResNet-50 и шаг за шагом переносили в него решения из трансформеров (Swin). Почти половина прироста — просто рецепт обучения; архитектурные шаги по отдельности дают десятые доли процента, вместе — свёрточную сеть не хуже трансформера при тех же вычислениях



Liu et al., A ConvNet for the 2020s, CVPR 2022, Table 10; у ResNet-50, ConvNeXt-T и Swin-T одинаковые ≈4.5 GFLOPs

https://arxiv.org/abs/2201.03545

Свёрточные сети сегодня

- Большие предобученные энкодеры картинок (CLIP, SigLIP, DINOv2) — трансформеры
- Свёртки остаются там, где малая обучающая выборка (встроенная локальность экономит примеры), мало вычислений (телефоны, встраиваемые устройства) или важна скорость (детекция на видео)
- Backbone для детекции и сегментации: ConvNeXt / ConvNeXt V2 (2023)
- Телефоны и встраиваемые устройства: EfficientNetV2 (2021), MobileNetV4 (2024)
- Детекция в реальном времени: семейство YOLO — целиком свёрточное и, вероятно, самое массовое в продакшене; трансформеры впереди по качеству и по zero-shot
- Медицинская сегментация — UNet (nnU-Net); диффузионные модели до 2023 (Stable Diffusion 1–XL) — UNet, дальше трансформеры (DiT, SD3, Flux)
- «ConvNets Match Vision Transformers at Scale» (DeepMind, 2023): при равных данных и бюджете вычислений свёрточные сети масштабируются так же, как ViT, — прежняя разница была в объёме предобучения и рецепте, а не в свёртках

https://arxiv.org/abs/2310.16764

Принципы дизайна

- Пространственное разрешение уменьшаем, число каналов увеличиваем — от 224×224×3 к 7×7×512 и дальше
- Маленькие свёртки 3×3 — стандарт для обычных (плотных) свёрток; depthwise дешёвые, поэтому ядро можно брать большим — 7×7 у ConvNeXt; 1×1 — для работы с каналами и сжатия
- Residual-связи и нормализация в каждом блоке — иначе глубокие сети не учатся
- Global average pooling вместо полносвязных слоёв в конце
- Depthwise separable — когда экономим FLOPs и память под веса (телефоны, встраиваемые устройства); на GPU плотные свёртки при равном качестве часто быстрее (RegNet, ResNet-RS)
- Рецепт обучения даёт не меньше, чем архитектура: сравнивать сети честно можно только при одинаковом рецепте
- ResNet-50 или ConvNeXt-T — типичный бейзлай; трансформеры — когда есть предобученная на больших данных модель (DINOv2, CLIP) или свои данные такого масштаба

Transfer learning

Перенос знаний

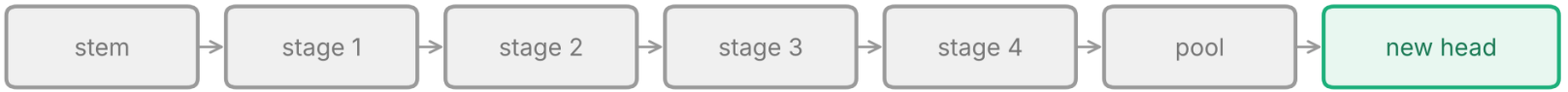
- ImageNet: много данных (которые сложно собрать!) и годы улучшений, не хотелось бы повторять это для каждой новой задачи
- Решение: взять сеть, обученную на ImageNet, и дообучить под свою задачу
  - признаки первых слоёв (края, текстуры) универсальны, чем глубже, тем специфичнее
- Откуда берут базовые модели сегодня: ImageNet-1k и -21k с учителем; предобучение без меток (DINOv2, MAE); картинка–текст (CLIP, SigLIP)

Три режима дообучения

- Чем меньше данных и чем ближе новая задача к исходной, тем меньше слоёв трогаем
- Более ранние слои — меньше learning rate

Linear probing

все веса заморожены, учится только новая голова (= линейная модель над фиксированными признаками)



сотни примеров, похожая задача

Partial fine-tuning

размораживаем последние стадии; чем ближе к входу, тем меньше learning rate



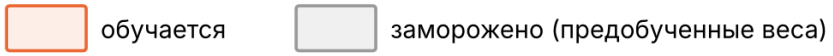
тысячи примеров

Full fine-tuning

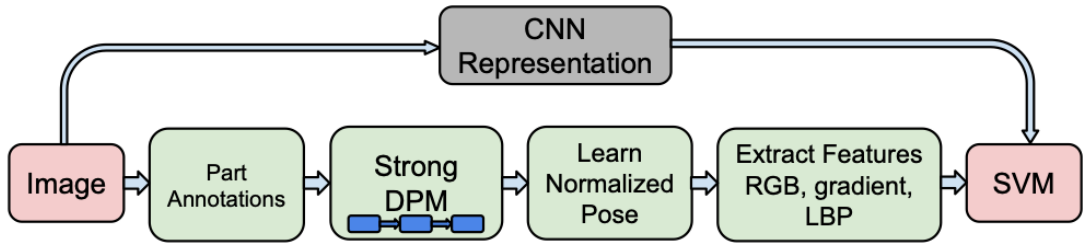
учатся все веса с маленьким learning rate (LP-FT: сначала голову при замороженной сети, потом всё)

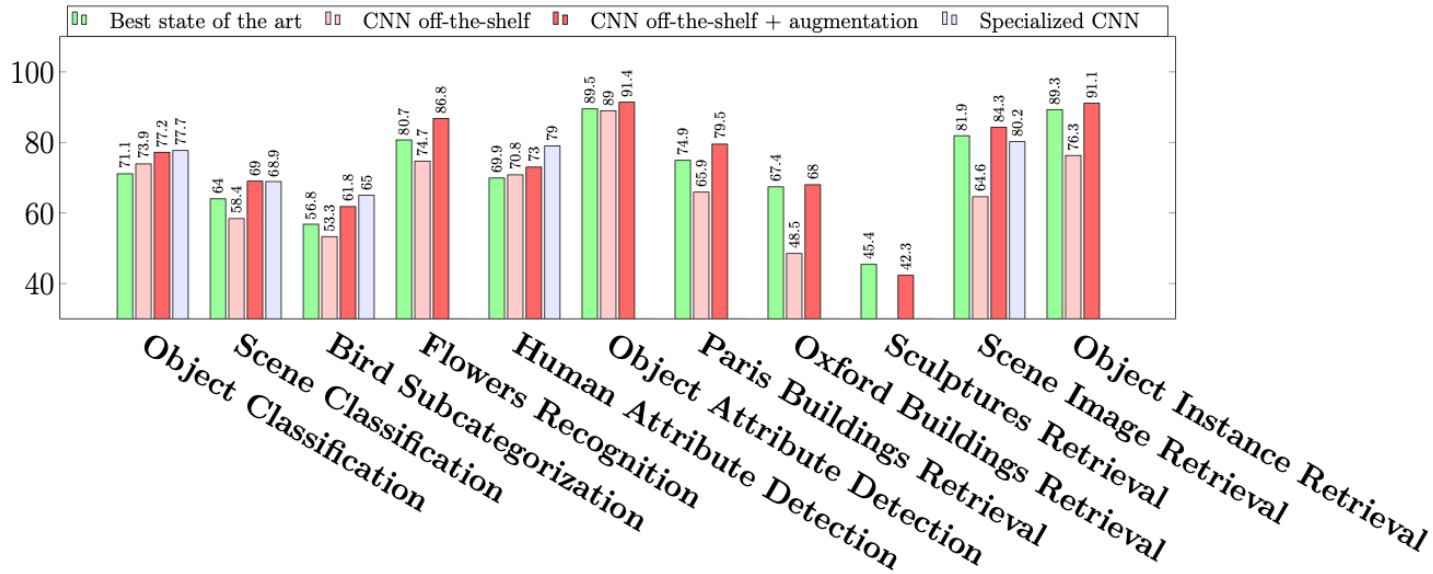


больше данных или другая задача



Представления с последних слоёв

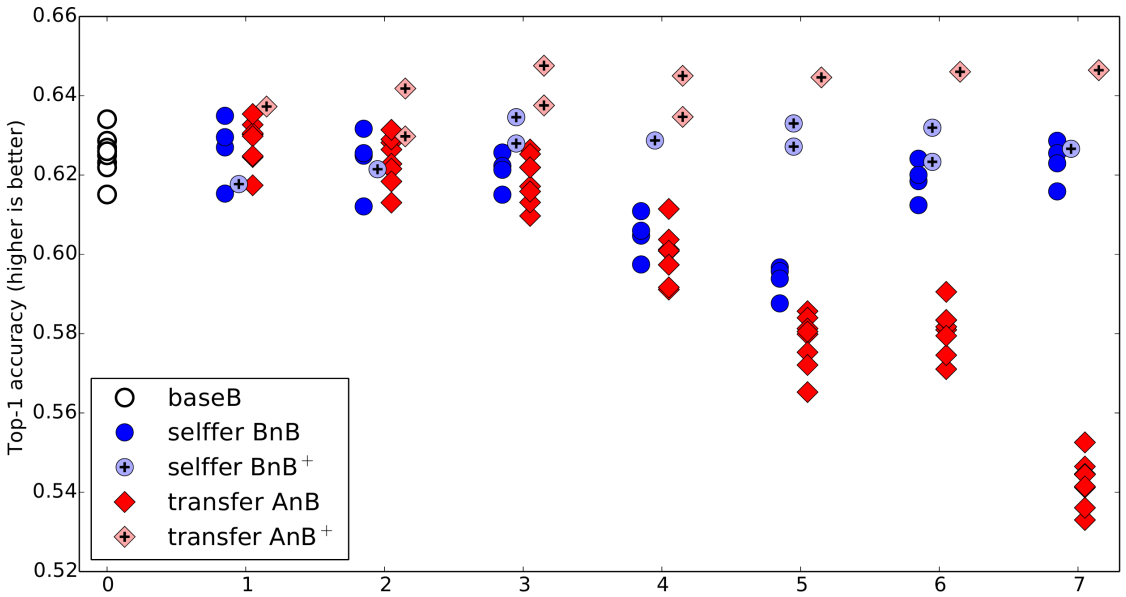




https://arxiv.org/abs/1403.6382

Какие слои переучивать

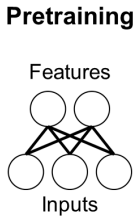
- Yosinski et al. 2014: копируем первые n слоёв из сети, обученной на других 500 классах ImageNet (ромбы), и учим остальное
- Surgical fine-tuning (Lee et al. 2023): какой слой трогать, зависит от разницы между задачами
  - шум или другая камера — первые слои
  - другая разметка — последние
- Правило «ближе к входу — меньше шаг» работает, когда отличаются метки; если отличается вход (камера, шум, домен), переучивать надо первые слои

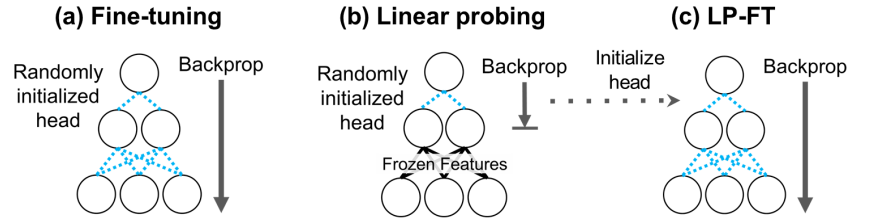


https://arxiv.org/abs/1411.1792

Полное дообучение портит признаки

- Kumar et al. 2022: предобученный экстрактор, новая задача, два теста — ID (то же распределение, что данные дообучения) и OOD (сдвиг: другая версия датасета, рисунки вместо фото)
- Три способа: (a) полное дообучение со случайной головой; (b) учим только голову; (c) LP-FT — сначала (b), потом (a)
- Механизм: случайная голова шлёт в признаки большой случайный градиент — предобученные признаки портятся там, где для ID это незаметно, а OOD на них держится. С готовой головой градиент в признаки мал, дообучение их лишь подправляет
- Приём: сначала несколько эпох только голова при замороженной сети, потом разморозить всё с маленьким шагом





	(a) Fine-tuning	(b) Linear probing	(c) LP-FT
ID test	85.1%	82.9%	**85.7%**
OOD test	59.3%	66.2%	**68.9%**

Average accuracies (10 distribution shifts)

https://arxiv.org/abs/2202.10054

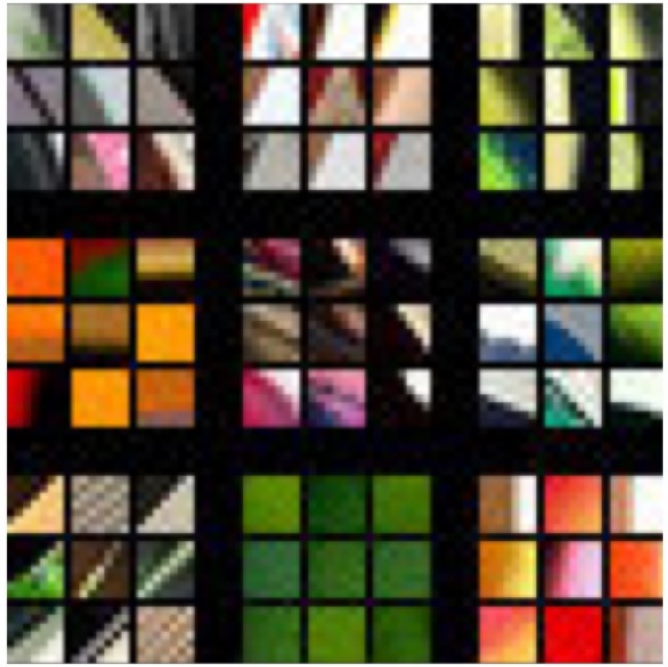
Интерпретация моделей

Что находят свёрточные сети?

- Можно для каждого фильтра найти кусочки картинок, дающие самый сильный отклик — сделаем это для AlexNet (Zeiler & Fergus, 2014)

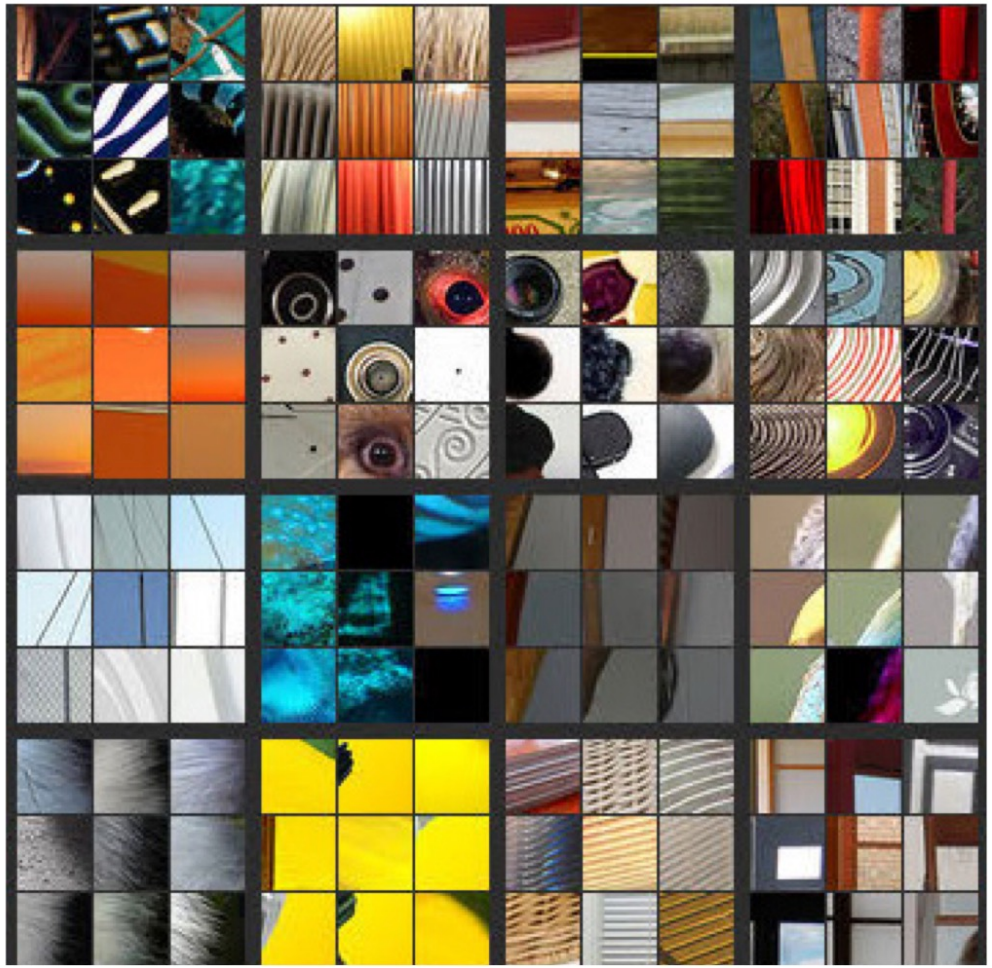
https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf

Слой 1



https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf

Слой 2



https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf

Слой 3



https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf

Слой 4



https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf

Слой 5



https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf

Максимизация вероятности класса

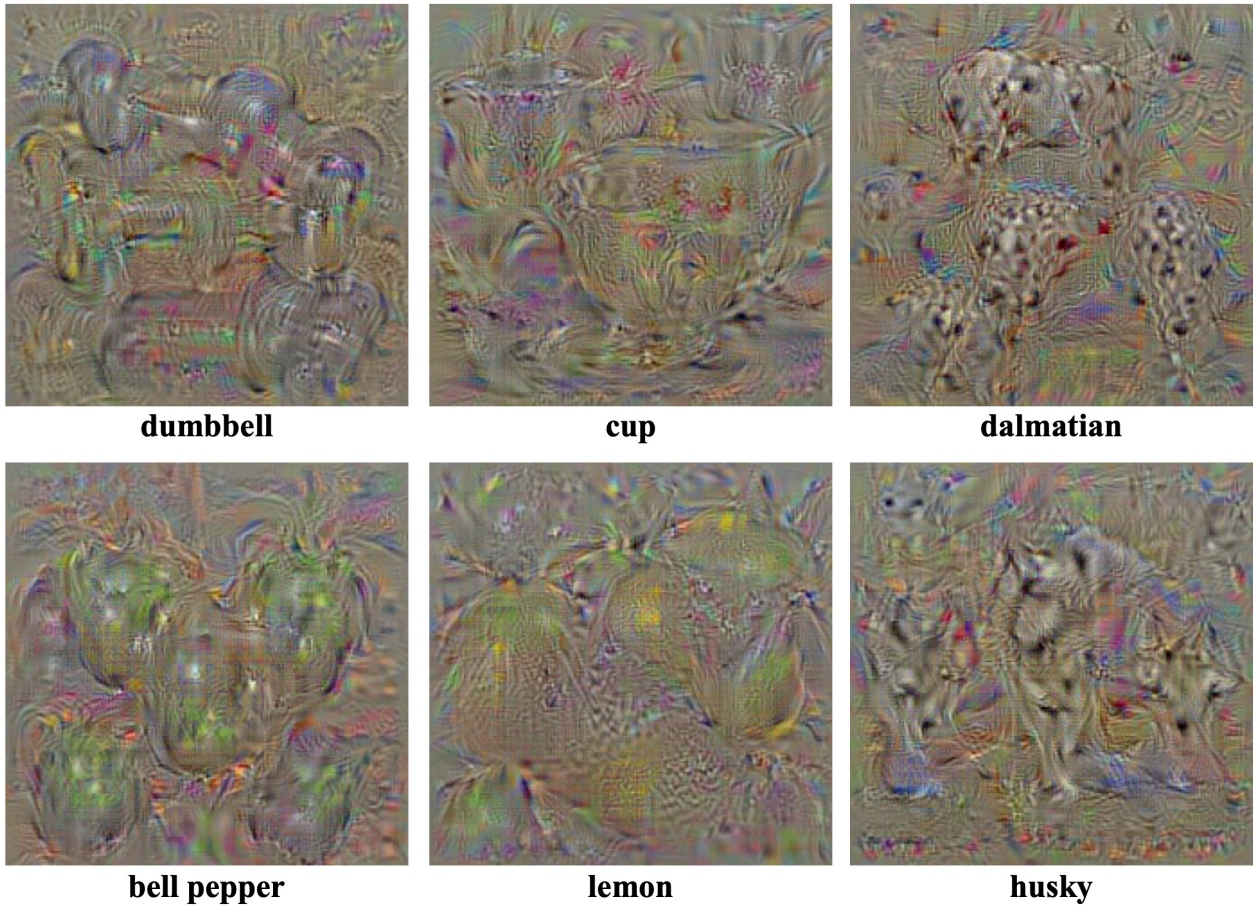
$$a_y(x) - \lambda \|x\|_2^2 \to \max_x$$

- Инициализируем картинку случайным шумом
- Ищем оптимальную для данного класса картинку градиентным спуском

- Без ограничений оптимизация находит состязательный шум, а не «образ класса» — картинки 2013 года серые и невнятные

https://arxiv.org/pdf/1312.6034.pdf

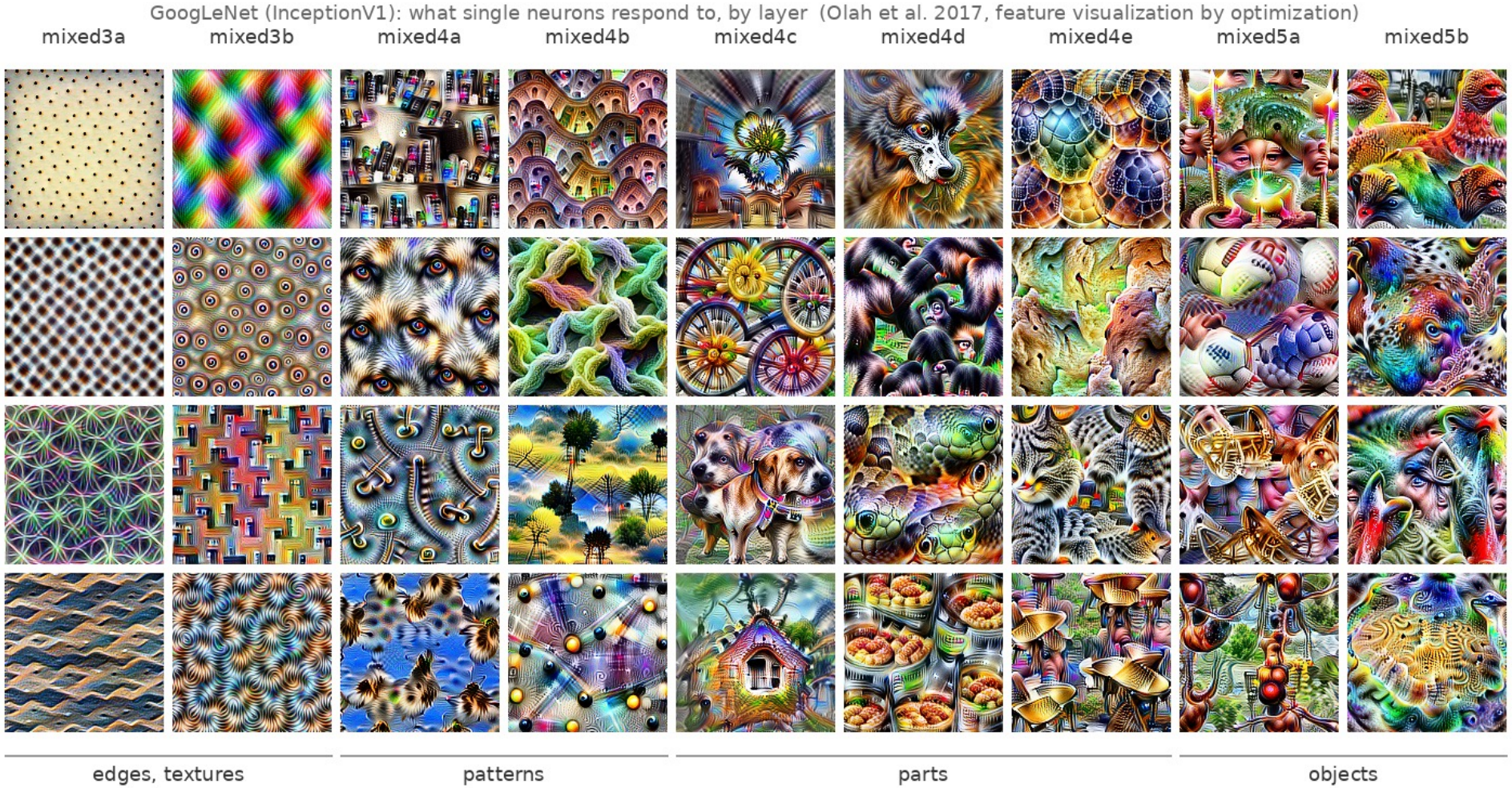
Максимизация вероятности класса



https://arxiv.org/pdf/1312.6034.pdf

Feature visualization (Olah et al., 2017)

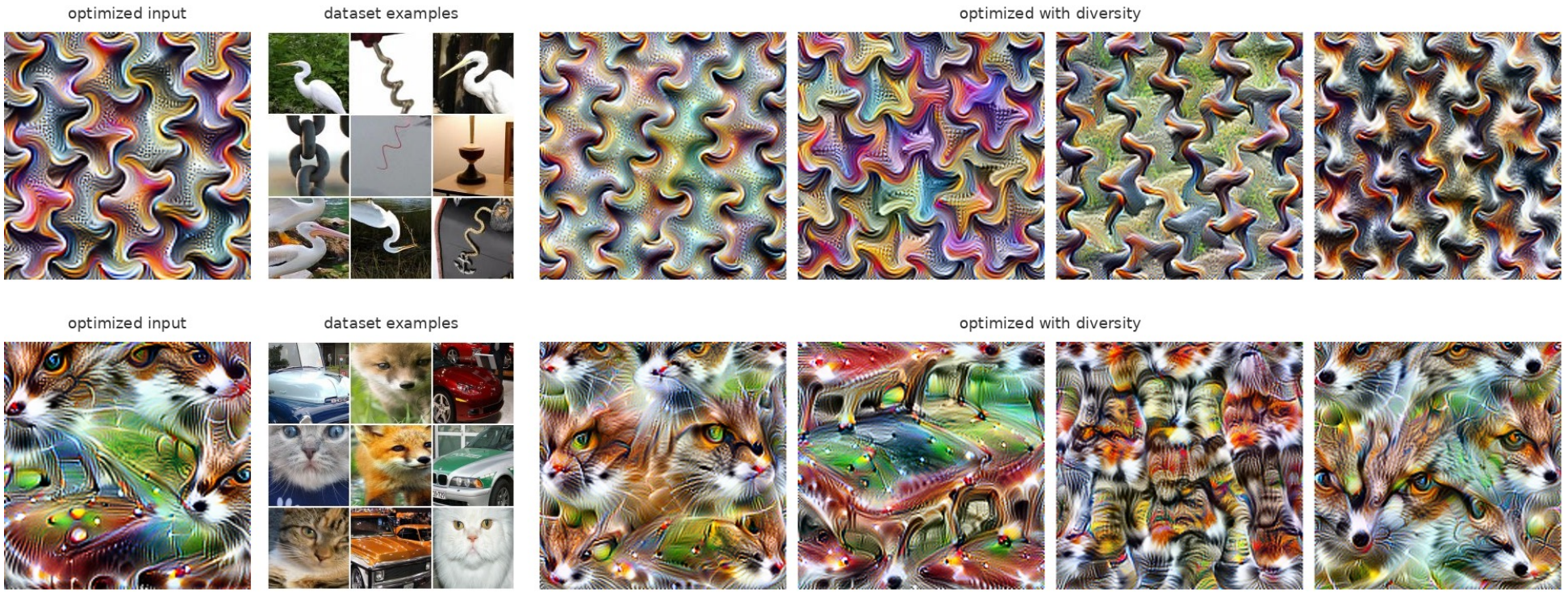
- Та же оптимизация по входу, но с регуляризацией: случайные сдвиги, повороты и масштаб на каждом шаге, параметризация в частотной области, декоррелированные цвета



https://distill.pub/2017/feature-visualization

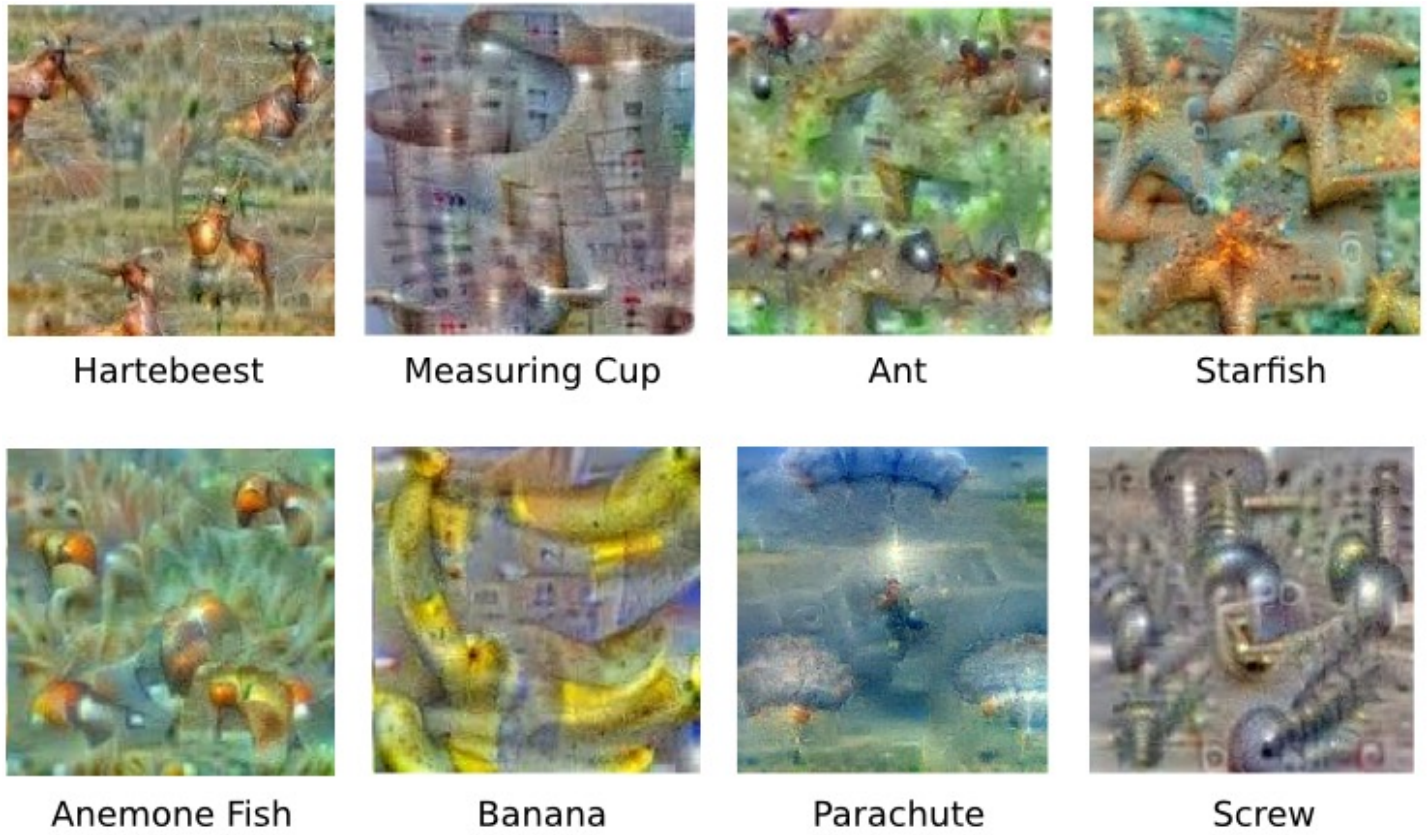
Feature visualization (Olah et al., 2017)

- Нейрон mixed4e:55 отвечает на кошачьи морды, передки машин и кошачьи лапы — нельзя утверждать, что он отвечает за одно понятие



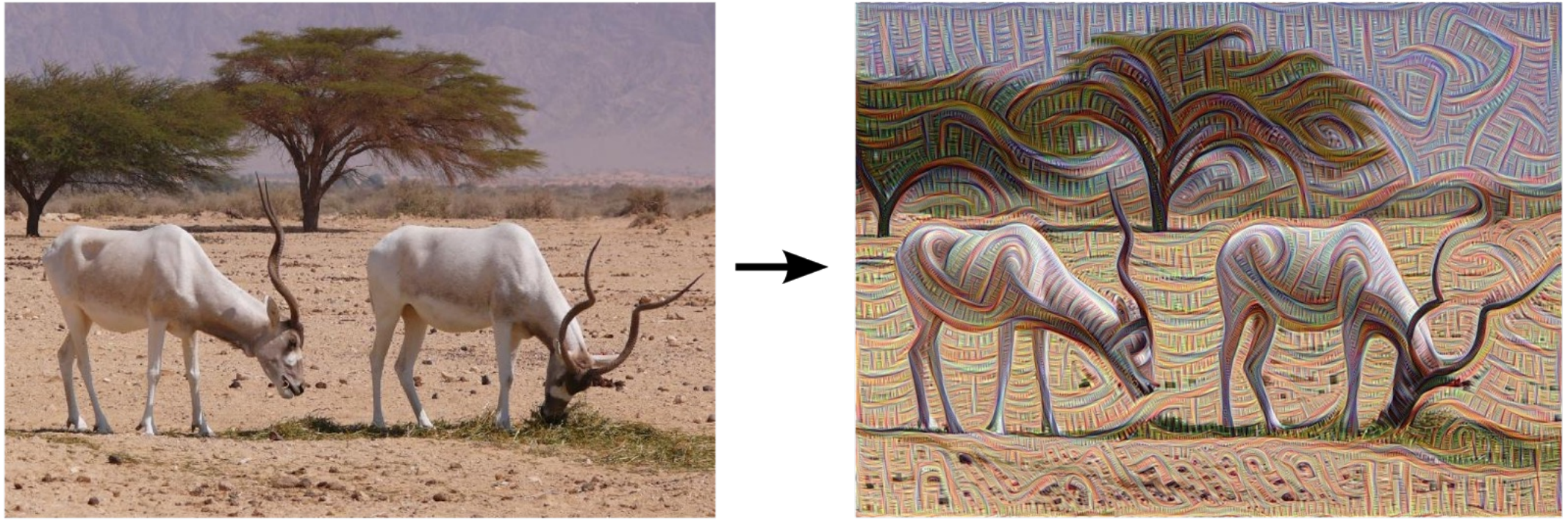
https://distill.pub/2017/feature-visualization

Максимизация вероятности класса



https://research.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html

Максимизация вероятности класса



https://research.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html

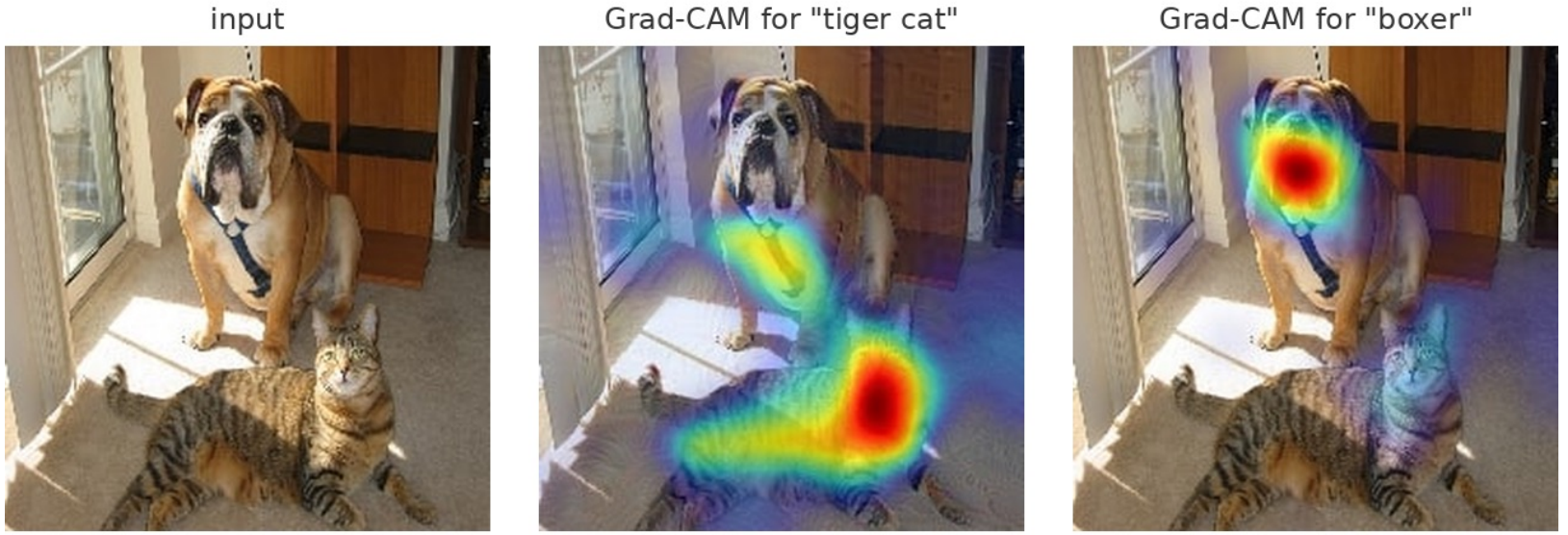
Максимизация вероятности класса



https://research.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html

Куда смотрела сеть: Grad-CAM

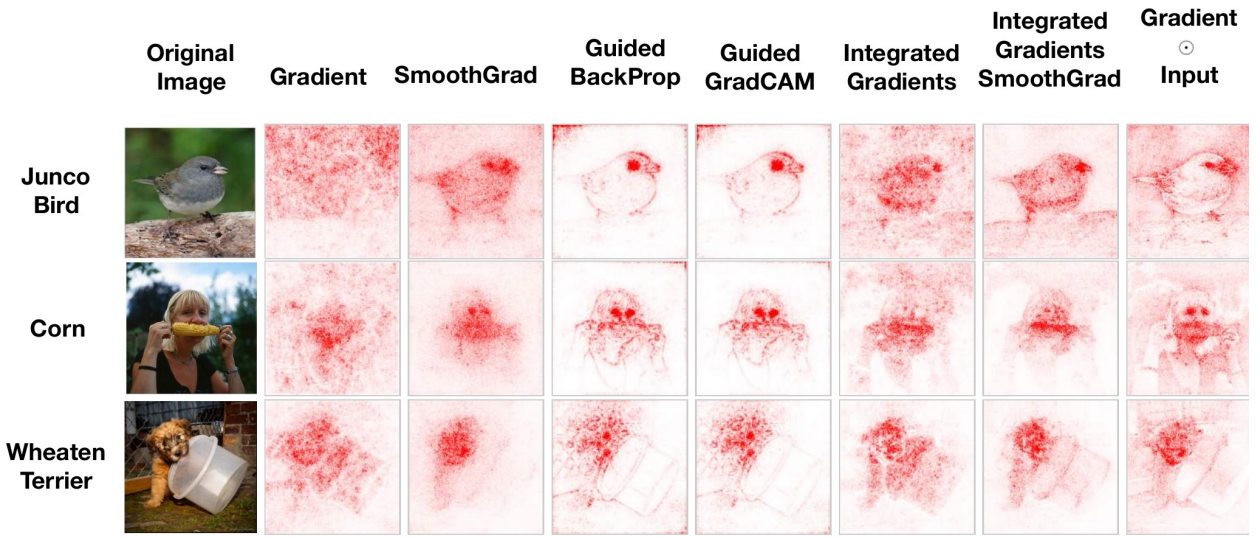
- Берём карты признаков последнего свёрточного слоя A^k (у ResNet-50 — 2048 карт 7×7) и логит класса y^c
- Шаг 1, вес канала: $\alpha_k = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial {A^k}_{ij}}$ — насколько канал k в среднем «голосует за» класс c
- Шаг 2, карта: $L^c = \mathrm{ReLU}\left(\sum_k \alpha_k A^k\right)$ — складываем карты с этими весами; ReLU оставляет только места, которые повышают логит класса
- Шаг 3: растягиваем 7×7 до размера картинки и накладываем тепловой картой
- Один прямой и один обратный проход; любая свёрточная сеть, без переобучения
- Зачем: проверить, что модель смотрит на объект, а не на фон; найти утечки (снег на всех фото волков); бесплатная сегментация через классификатор



https://arxiv.org/abs/1610.02391

Карты важности по пикселям

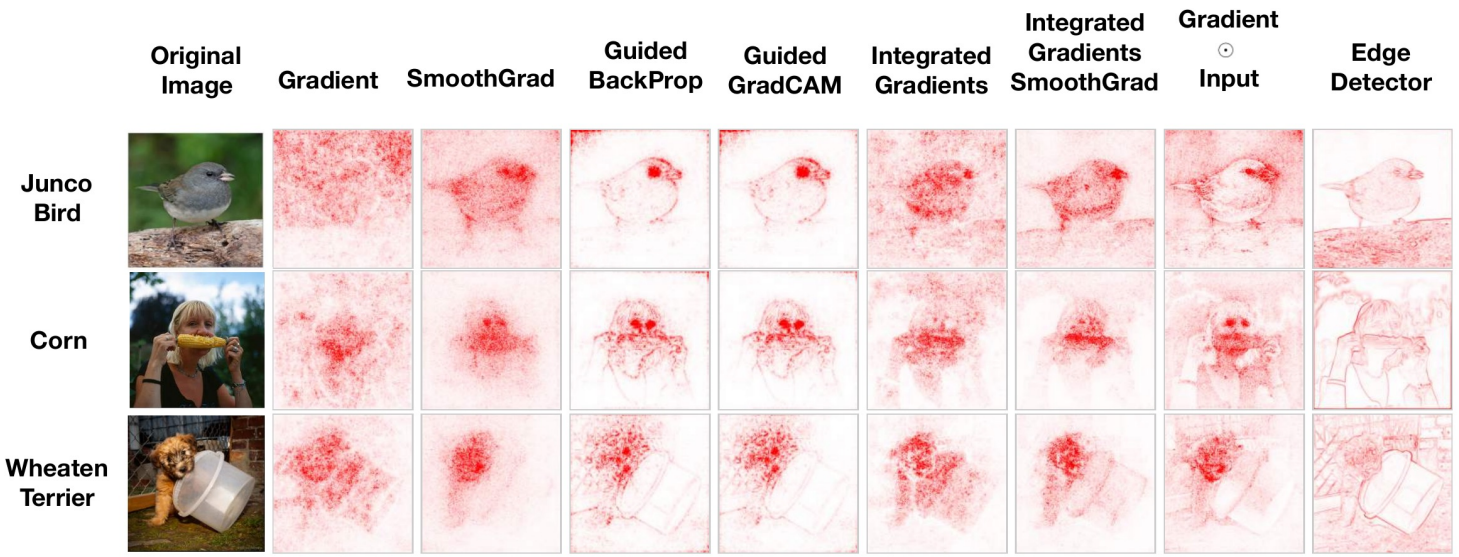
- Gradient / saliency (Simonyan et al., 2013): карта = $\left|\frac{\partial y^c}{\partial x}\right|$ — модуль градиента логита класса по пикселям: насколько выход чувствителен к каждому пикселю
- SmoothGrad (2017): тот же градиент, усреднённый по десяткам копий картинки с добавленным шумом
- Guided Backprop (Springenberg et al., 2015): обратный проход, где через каждый ReLU пропускают только положительные градиенты — карты чёткие, как контурный рисунок
- Integrated Gradients (2017): градиент интегрируют вдоль пути от чёрной картинки к данной — чувствительность накапливается по всему пути
- Grad-CAM — по каналам последнего свёрточного слоя; Guided Grad-CAM — произведение Guided Backprop и растянутого Grad-CAM



https://arxiv.org/abs/1810.03292

Проверка на вменяемость

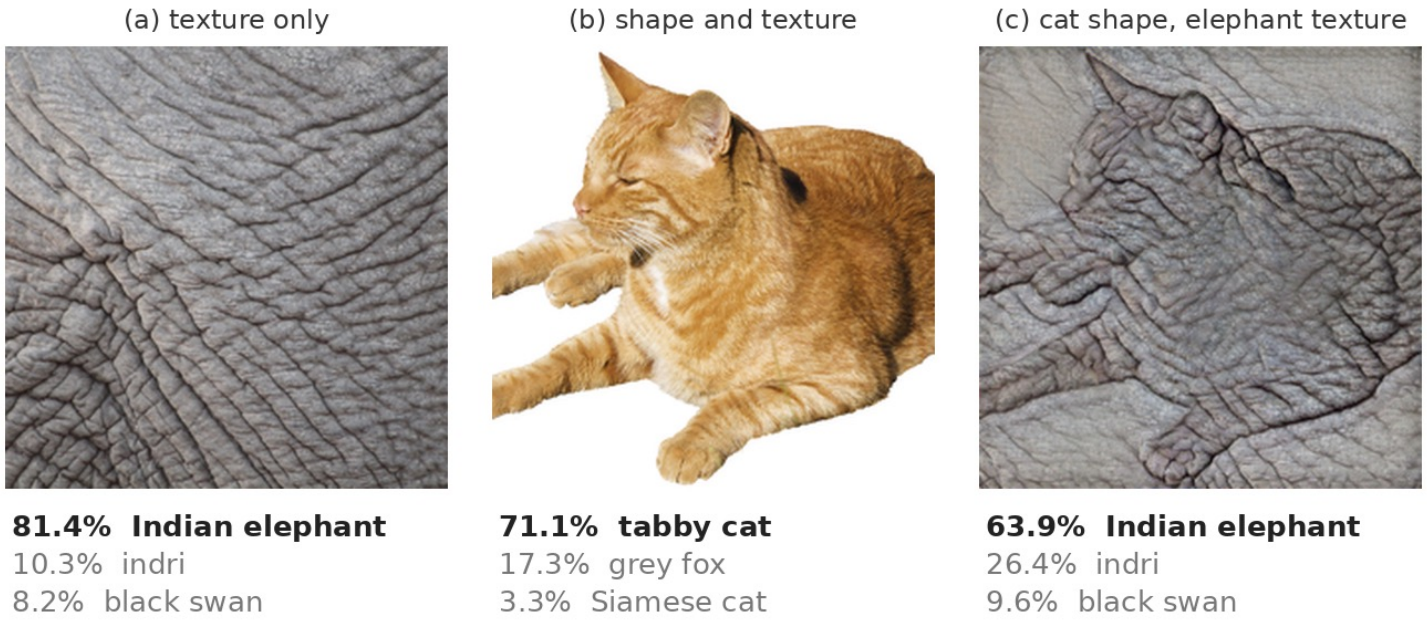
- Тест 1: заменяем обученные веса случайными и пересчитываем карту. Если карта не изменилась — метод не зависит от модели и ничего о ней не сообщает
- Тест 2: обучаем сеть на случайно перемешанных метках. Если карта такая же, как у нормальной сети, — метод не зависит от того, что сеть выучила
- Результат: градиент и Grad-CAM тесты проходят — их карты меняются. Guided Backprop и Guided Grad-CAM — нет, их карты почти совпадают с выходом детектора краёв, который модели вообще не видел (правая колонка)



https://arxiv.org/abs/1810.03292

Текстура или форма?

- Geirhos et al. 2019: кошка с текстурой слона для ResNet-50 — слон. Люди в таком конфликте выбирают форму в ~95% случаев, ImageNet-сети — текстуру в 70–80%
- Сеть учит самый дешёвый признак, которого хватает для ImageNet, — shortcut learning. Обучение на Stylized-ImageNet (случайный стиль у каждой картинки) сдвигает к форме и даёт устойчивость к искажениям и +4.4 mAP в детекции



Top-3 predictions of an ImageNet-trained ResNet-50 (Geirhos et al., ICLR 2019)

https://arxiv.org/abs/1811.12231